

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB RECORD

Bio Inspired Systems (23CS5BSBIS)

Submitted by

PRATHITH SURESH RAO (1BM23CS247)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Sep-2025 to Jan-2026

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “Bio Inspired Systems (23CS5BSBIS)” carried out by **PRATHITH SURESH RAO (1BM23CS247)**, who is Bonafide student of **B.M.S. College of Engineering**. It is in partial fulfilment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements of the above-mentioned subject and the work prescribed for the said degree.

PROF.SWATHI SRIDHARAN Assistant Professor Department of CSE, BMSCE	Dr. KAVITHA SOODA Professor & HOD Department of CSE, BMSCE
--	--

Index

Sl. No	Date	Experiment Title	Page No.
1	28-08-2025	GENETIC ALGORITHM	1
2	04-09-2025	GENE EXPRESSION ALGORITHM	4
3	11-09-2025	PARTICLE SWARM OPTIMIZATION	13
4	09-10-2025	ANT COLONY	24
5	16-10-2025	CUCKOO SEARCH	37
6	23-10-2025	GREY WOLF OPTIMIZER	43
7	30-10-2025	PARALLEL CELLULAR ALGORITHM	47

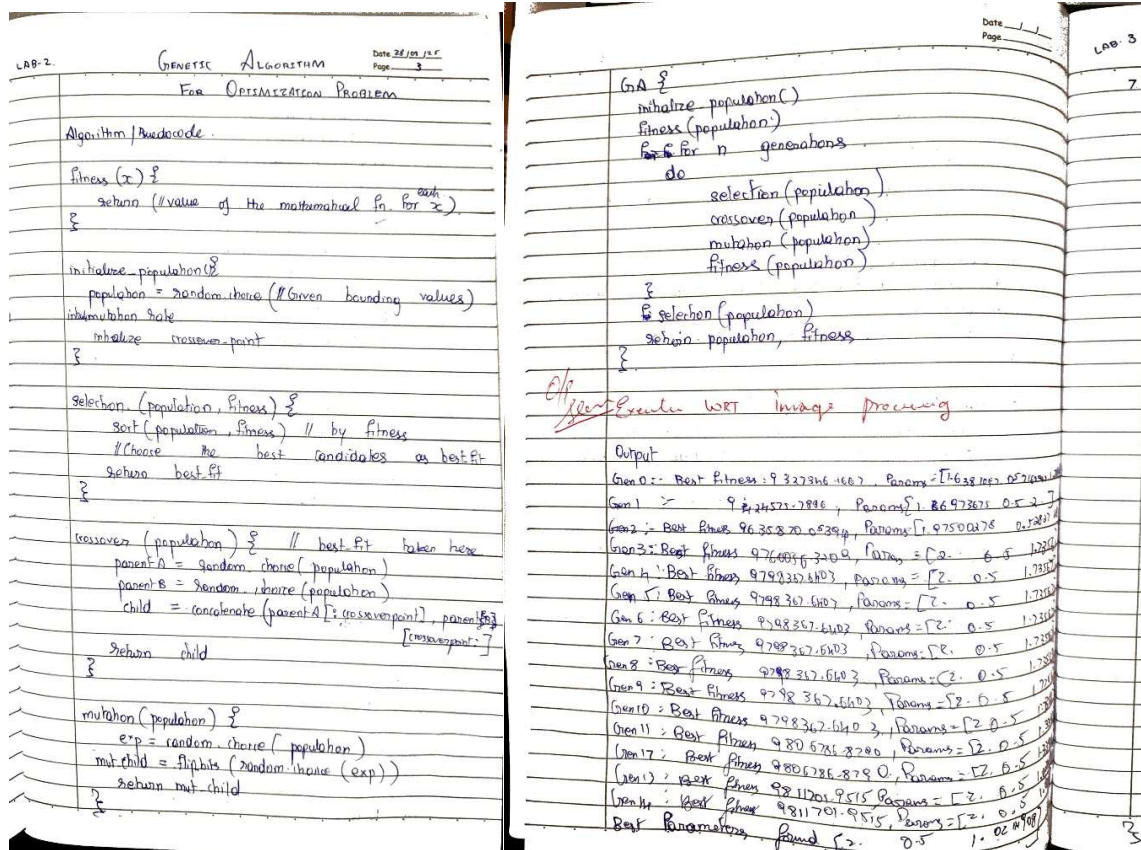
Github Link:

https://github.com/Prathith-Rao/1BM23CS247_5/tree/main/BIS_LAB

PROGRAM 1: GENETIC ALGORITHM FOR OPTIMIZATION PROBLEMS

Genetic Algorithms (GA) are inspired by the process of natural selection and genetics, where the fittest individuals are selected for reproduction to produce the next generation. GAs are widely used for solving optimization and search problems. Implement a Genetic Algorithm using Python to solve a basic optimization problem, such as finding the maximum value of a mathematical function.

Algorithm:



Code:

```
import random
import numpy as np
from PIL import Image, ImageEnhance
from skimage.metrics import structural_similarity as ssim
from skimage.filters import sobel
from skimage import img_as_float
# ----- Parameters -----
POP_SIZE = 20
N_GEN = 15
MUT_RATE = 0.3
ELITE = 2 # keep best individuals each gen

# Load image
original = Image.open("input.jpg").convert("RGB")
```

```

original_np = np.array(original)

# If you have a reference image, load it
try:
    target = Image.open("target.jpg").convert("RGB")
    target_np = np.array(target)
    USE_REFERENCE = True
except:
    USE_REFERENCE = False

# ----- Apply Adjustments -----
def apply_adjustments(params):
    b, c, s = params
    img = original.copy()
    img = ImageEnhance.Brightness(img).enhance(b)
    img = ImageEnhance.Contrast(img).enhance(c)
    img = ImageEnhance.Sharpness(img).enhance(s)
    return img

# ----- Fitness Function -----
def fitness(params):
    img = apply_adjustments(params)
    img_np = np.array(img)

    if USE_REFERENCE:
        return ssim(target_np, img_np, channel_axis=2)
    else:
        gray = img.convert("L")
        gray_np = img_as_float(np.array(gray))
        entropy = -np.sum(gray_np * np.log2(gray_np + 1e-10))
        edge_strength = np.mean(sobel(gray_np))
        return entropy + edge_strength

# ----- GA Core -----
def init_population(size):
    return [np.array([
        random.uniform(0.5, 2.0), # Brightness
        random.uniform(0.5, 2.0), # Contrast
        random.uniform(0.5, 2.0) # Sharpness
    ]) for _ in range(size)]

def crossover(p1, p2):
    alpha = random.random()
    return alpha * p1 + (1 - alpha) * p2

def mutate(ind):
    if random.random() < MUT_RATE:
        ind += np.random.normal(0, 0.2, size=3)
        ind = np.clip(ind, 0.5, 2.0)
    return ind

def select(pop, fitnesses):
    i, j = random.sample(range(len(pop)), 2)
    return pop[i] if fitnesses[i] > fitnesses[j] else pop[j]

# ----- Run GA -----
def run_ga():

```

```

pop = init_population(POP_SIZE)

for gen in range(N_GEN):
    fitnesses = [fitness(ind) for ind in pop]
    ranked = sorted(zip(pop, fitnesses), key=lambda x: x[1], reverse=True)
    best_ind, best_fit = ranked[0]

    print(f'Gen {gen}: Best fitness = {best_fit:.4f}, Params = {best_ind}')

    new_pop = [ind.copy() for ind, _ in ranked[:ELITE]]

    while len(new_pop) < POP_SIZE:
        p1, p2 = select(pop, fitnesses), select(pop, fitnesses)
        child = crossover(p1, p2)
        child = mutate(child)
        new_pop.append(child)

    pop = new_pop

best_img = apply_adjustments(best_ind)
best_img.save("optimized.jpg")
print("Best parameters found:", best_ind)
display(best_img) # Show inside Jupyter

# Run it
run_ga()

```

Ouput:

```

Gen 0: Best fitness = 9327346.1667, Params = [1.63810974 0.57168993 1.75003555]
Gen 1: Best fitness = 9424575.7286, Params = [1.36973675 0.5      2.      ]
Gen 2: Best fitness = 9635870.0394, Params = [1.97500278 0.52827145 1.4910719 ]
Gen 3: Best fitness = 9760056.3209, Params = [1.82492376 0.5      1.51500324]
Gen 4: Best fitness = 9798367.6403, Params = [2.      0.5      1.73567691]
Gen 5: Best fitness = 9798367.6403, Params = [2.      0.5      1.73567691]
Gen 6: Best fitness = 9798367.6403, Params = [2.      0.5      1.73567691]
Gen 7: Best fitness = 9798367.6403, Params = [2.      0.5      1.73567691]
Gen 8: Best fitness = 9798367.6403, Params = [2.      0.5      1.73567691]
Gen 9: Best fitness = 9798367.6403, Params = [2.      0.5      1.73567691]
Gen 10: Best fitness = 9806786.8790, Params = [2.      0.5      1.33926455]
Gen 11: Best fitness = 9806786.8790, Params = [2.      0.5      1.33926455]
Gen 12: Best fitness = 9811701.9515, Params = [2.      0.5      1.02414908]
Gen 13: Best fitness = 9811701.9515, Params = [2.      0.5      1.02414908]
Gen 14: Best fitness = 9811701.9515, Params = [2.      0.5      1.02414908]
Best parameters found: [2.      0.5      1.02414908]

```

PROGRAM 2 : OPTIMIZATION VIA GENE EXPRESSION ALGORITHMS

Gene Expression Algorithms (GEA) are inspired by the biological process of gene expression in living organisms. This process involves the translation of genetic information encoded in DNA into functional proteins. In GEA, solutions to optimization problems are encoded in a manner similar to genetic sequences. The algorithm evolves these solutions through selection, crossover, mutation, and gene expression to find optimal or near-optimal solutions. GEA is effective for solving complex optimization problems in various domains, including engineering, data analysis, and machine learning.

Algorithm:

LAB 3

Date: 21/09/25

Page: 4

7 OPTIMIZATION VIA GENE EXPRESSION ALGORITHMS

Algorithm / Pseudocode

```

Func Fitness (genotype) {
    // Express genotype into solution phenotype
    phenotype = gene_expression_algo (genotype)
    fit = compute_value_for_fitness (phenotype)
    return fit
}

Initialize_population {
    population = input ("Initial genotype assumption")
    mutation_rate = input()
    selection_cutoff = input()
    (crossover_point = input())
    (max_generations = input())
}

Selection (population, fitness) {
    Sort (population, fitness) // by fitness
    new_population = population [0:selection_cutoff] // best fit
    return new_population
}

reproduce (population) {
    for i in range (0, len(population)) {
        parent A = random.choice (population)
        parent B = random.choice (population)
        child = crossover (parent A, crossover_point, parent B, crossover_point)
        new_population.append (child)
    }
    return new_population
}

mutation (population) {
    for i in range (0, len(population)) {
        exp = random.choice (population)
        mut_child = flip_complement_for_bit_in_exp [i:2]
        population[exp] = mut_child
    }
    return population
}

gene = Func gene_expression_algo (genotype) {
    // genotype = { chromosome, protein set, terminal set }
    expression_stack = []
    for gene in chromosome {
        if gene in terminal_set {
            gene = push_expression_stack, push (gene)
        } else if gene in function_set {
            operands [] = pop_expression_stack, pop ()
            new_expression = operands [0] gene operands [1]
            expression_stack, push (new_expression)
        }
    }
    phenotype = expression_stack.pop ()
    return phenotype // solution
}

Func GEA {
    initialize_population ()
    fitness (population)
    for i in range (0, generations) {
        pop = selection (population)
        pop = crossover (population)
        pop = mutation (population)
        gene_phenotype = gene_expression_algo (population)
    }
}

```

Output Image

Gene 1

Gene 2

Gene 3

Gene 4

Gene 5

Gene 6

Gene 7

Gene 8

Gene 9

Gene 10

Gene 11

Gene 12

Gene 13

Gene 14

Gene 15

Gene 16

Gene 17

Gene 18

Gene 19

Gene 20

Gene 21

Gene 22

Gene 23

Gene 24

Gene 25

Gene 26

Gene 27

Gene 28

Gene 29

Gene 30

Gene 31

Gene 32

Gene 33

Gene 34

Gene 35

Gene 36

Gene 37

Gene 38

Gene 39

Gene 40

Gene 41

Gene 42

Gene 43

Gene 44

Gene 45

Gene 46

Gene 47

Gene 48

Gene 49

Gene 50

Gene 51

Gene 52

Gene 53

Gene 54

Gene 55

Gene 56

Gene 57

Gene 58

Gene 59

Gene 60

Gene 61

Gene 62

Gene 63

Gene 64

Gene 65

Gene 66

Gene 67

Gene 68

Gene 69

Gene 70

Gene 71

Gene 72

Gene 73

Gene 74

Gene 75

Gene 76

Gene 77

Gene 78

Gene 79

Gene 80

Gene 81

Gene 82

Gene 83

Gene 84

Gene 85

Gene 86

Gene 87

Gene 88

Gene 89

Gene 90

Gene 91

Gene 92

Gene 93

Gene 94

Gene 95

Gene 96

Gene 97

Gene 98

Gene 99

Gene 100

Gene 101

Gene 102

Gene 103

Gene 104

Gene 105

Gene 106

Gene 107

Gene 108

Gene 109

Gene 110

Gene 111

Gene 112

Gene 113

Gene 114

Gene 115

Gene 116

Gene 117

Gene 118

Gene 119

Gene 120

Gene 121

Gene 122

Gene 123

Gene 124

Gene 125

Gene 126

Gene 127

Gene 128

Gene 129

Gene 130

Gene 131

Gene 132

Gene 133

Gene 134

Gene 135

Gene 136

Gene 137

Gene 138

Gene 139

Gene 140

Gene 141

Gene 142

Gene 143

Gene 144

Gene 145

Gene 146

Gene 147

Gene 148

Gene 149

Gene 150

Gene 151

Gene 152

Gene 153

Gene 154

Gene 155

Gene 156

Gene 157

Gene 158

Gene 159

Gene 160

Gene 161

Gene 162

Gene 163

Gene 164

Gene 165

Gene 166

Gene 167

Gene 168

Gene 169

Gene 170

Gene 171

Gene 172

Gene 173

Gene 174

Gene 175

Gene 176

Gene 177

Gene 178

Gene 179

Gene 180

Gene 181

Gene 182

Gene 183

Gene 184

Gene 185

Gene 186

Gene 187

Gene 188

Gene 189

Gene 190

Gene 191

Gene 192

Gene 193

Gene 194

Gene 195

Gene 196

Gene 197

Gene 198

Gene 199

Gene 200

Gene 201

Gene 202

Gene 203

Gene 204

Gene 205

Gene 206

Gene 207

Gene 208

Gene 209

Gene 210

Gene 211

Gene 212

Gene 213

Gene 214

Gene 215

Gene 216

Gene 217

Gene 218

Gene 219

Gene 220

Gene 221

Gene 222

Gene 223

Gene 224

Gene 225

Gene 226

Gene 227

Gene 228

Gene 229

Gene 230

Gene 231

Gene 232

Gene 233

Gene 234

Gene 235

Gene 236

Gene 237

Gene 238

Gene 239

Gene 240

Gene 241

Gene 242

Gene 243

Gene 244

Gene 245

Gene 246

Gene 247

Gene 248

Gene 249

Gene 250

Gene 251

Gene 252

Gene 253

Gene 254

Gene 255

Gene 256

Gene 257

Gene 258

Gene 259

Gene 260

Gene 261

Gene 262

Gene 263

Gene 264

Gene 265

Gene 266

Gene 267

Gene 268

Gene 269

Gene 270

Gene 271

Gene 272

Gene 273

Gene 274

Gene 275

Gene 276

Gene 277

Gene 278

Gene 279

Gene 280

Gene 281

Gene 282

Gene 283

Gene 284

Gene 285

Gene 286

Gene 287

Gene 288

Gene 289

Gene 290

Gene 291

Gene 292

Gene 293

Gene 294

Gene 295

Gene 296

Gene 297

Gene 298

Gene 299

Gene 300

Gene 301

Gene 302

Gene 303

Gene 304

Gene 305

Gene 306

Gene 307

Gene 308

Gene 309

Gene 310

Gene 311

Gene 312

Gene 313

Gene 314

Gene 315

Gene 316

Gene 317

Gene 318

Gene 319

Gene 320

Gene 321

Gene 322

Gene 323

Gene 324

Gene 325

Gene 326

Gene 327

Gene 328

Gene 329

Gene 330

Gene 331

Gene 332

Gene 333

Gene 334

Gene 335

Gene 336

Gene 337

Gene 338

Gene 339

Gene 340

Gene 341

Gene 342

Gene 343

Gene 344

Gene 345

Gene 346

Gene 347

Gene 348

Gene 349

Gene 350

Gene 351

Gene 352

Gene 353

Gene 354

Gene 355

Gene 356

Gene 357

Gene 358

Gene 359

Gene 360

Gene 361

Gene 362

Gene 363

Gene 364

Gene 365

Gene 366

Gene 367

Gene 368

Gene 369

Gene 370

Gene 371

Gene 372

Gene 373

Gene 374

Gene 375

Gene 376

Gene 377

Gene 378

Gene 379

Gene 380

Gene 381

Gene 382

Gene 383

Gene 384

Gene 385

Gene 386

Gene 387

Gene 388

Gene 389

Gene 390

Gene 391

Gene 392

Gene 393

Gene 394

Gene 395

Gene 396

Gene 397

Gene 398

Gene 399

Gene 400

Gene 401

Gene 402

Gene 403

Gene 404

Gene 405

Gene 406

Gene 407

Gene 408

Gene 409

Gene 410

Gene 411

Gene 412

Gene 413

Gene 414

Gene 415

Gene 416

Gene 417

Gene 418

Gene 419

Gene 420

Gene 421

Gene 422

Gene 423

Gene 424

Gene 425

Gene 426

Gene 427

Gene 428

Gene 429

Gene 430

Gene 431

Gene 432

Gene 433

Gene 434

Gene 435

Gene 436

Gene 437

Gene 438

Gene 439

Gene 440

Gene 441

Gene 442

Gene 443

Gene 444

Gene 445

Gene 446

Gene 447

Gene 448

Gene 449

Gene 450

Gene 451

Gene 452

Gene 453

Gene 454

Gene 455

Gene 456

Gene 457

Gene 458

Gene 459

Gene 460

Gene 461

Gene 462

Gene 463

Gene 464

Gene 465

Gene 466

Gene 467

Gene 468

Gene 469

Gene 470

Gene 471

Gene 472

Gene 473

Gene 474

Gene 475

Gene 476

Gene 477

Gene 478

Gene 479

Gene 480

Gene 481

Gene 482

Gene 483

Gene 484

Gene 485

Gene 486

Gene 487

Gene 488

Gene 489

Gene 490

Gene 491

Gene 492

Gene 493

Gene 494

Gene 495

Gene 496

Gene 497

Gene 498

Gene 499

Gene 500

Gene 501

Gene 502

Gene 503

Gene 504

Gene 505

Gene 506

Gene 507

Gene 508

Gene 509

Gene 510

Gene 511

Gene 512

Gene 513

Gene 514

Gene 515

Gene 516

Gene 517

Gene 518

Gene 519

Gene 520

Gene 521

Gene 522

Gene 523

Gene 524

Gene 525

Gene 526

Gene 527

Gene 528

Gene 529

Gene 530

Gene 531

Gene 532

Gene 533

Gene 534

Gene 535

Gene 536

Gene 537

Gene 538

Gene 539

Gene 540

Gene 541

Gene 542

Gene 543

Gene 544

Gene 545

Gene 546

Gene 547

Gene 548

Gene 549

Gene 550

Gene 551

Gene 552

Gene 553

Gene 554

Gene 555

Gene 556

Gene 557

Gene 558

Gene 559

Gene 560

Gene 561

Gene 562

Gene 563

Gene 564

Gene 565

Gene 566

Gene 567

Gene 568

Gene 569

Gene 570

Gene 571

Gene 572

Gene 573

Gene 574

Gene 575

Gene 576

Gene 577

Gene 578

Gene 579

Gene 580

Gene 581

Gene 582

Gene 583

Gene 584

Gene 585

Gene 586

Gene 587

Gene 588

Gene 589

Gene 590

Gene 591

Gene 592

Gene 593

Gene 594

Gene 595

Gene 596

Gene 597

Gene 598

Gene 599

Gene 600

Gene 601

Gene 602

Gene 603

Gene 604

Gene 605

Gene 606

Gene 607

Gene 608

Gene 609

Gene 610

Gene 611

Gene 612

Gene 613

Gene 614

Gene 615

Gene 616

Gene 617

Gene 618

Gene 619

Gene 620

Gene 621

Gene 622

Gene 623

Gene 624

Gene 625

Gene 626

Gene 627

Gene 628

Gene 629

Gene 630

Gene 631

Gene 632

Gene 633

Gene 634

Gene 635

Gene 636

Gene 637

Gene 638

Gene 639

Gene 640

Gene 641

Gene 642

Gene 643

Gene 644

Gene 645

Gene 646

Gene 647

Gene 648

Gene 649

Gene 650

Gene 651

Gene 652

Gene 653

Gene 654

Gene 655

Gene 656

Gene 657

Gene 658

Gene 659

Gene 660

Gene 661

Gene 662

Gene 663

Gene 664

Gene 665

Gene 666

Gene 667

Gene 668

Gene 669

Gene 670

Gene 671

Gene 672

Gene 673

Gene 674

Gene 675

Gene 676

Gene 677

Gene 678

Gene 679

Gene 680

Gene 681

Gene 682

Gene 683

Gene 684

Gene 685

Gene 686

Gene 687

Gene 688

Gene 689

Gene 690

Gene 691

Gene 692

Gene 693

Gene 694

Gene 695

Gene 696

Gene 697

Gene 698

Gene 699

Gene 700

Gene 701

Gene 702

Gene 703

Gene 704

Gene 705

Gene 706

Gene 707

Gene 708

Gene 709

Gene 710

Gene 711

Gene 712

Gene 713

Gene 714

Gene 715

Gene 716

Gene 717

Gene 718

Gene 719

Gene 720

Gene 721

Gene 722

Gene 723

Gene 724

Gene 725

Gene 726

Gene 727

Gene 728

Gene 729

Gene 730

Gene 731

Gene 732

Gene 733

Gene 734

Gene 735

Gene 736

Gene 737

Gene 738

Gene 739

Gene 740

Gene 741

Gene 742

Gene 743

Gene 744

Gene 745

Gene 746

Gene 747

Gene 748

Gene 749

Gene 750

Gene 751

Gene 752

Gene 753

Gene 754

Gene 755

Gene 756

Gene 757

Gene 758

Gene 759

Gene 760

Gene 761

Gene 762

Gene 763

Gene 764

Gene 765

Gene 766

Gene 767

Gene 768

Gene 769

Gene 770

Gene 771

Gene 772

Gene 773

Gene 774

Gene 775

Gene 776

Gene 777

Gene 778

Gene 779

Gene 780

Gene 781

Gene 782

Gene 783

Gene 784

Gene 785

Gene 786

Gene 787

Gene 788

Gene 789

Gene 790

Gene 791

Gene 792

Gene 793

Gene 794

Gene 795

Gene 796

Gene 797

Gene 798

Gene 799

Gene 800

Gene 801

Gene 802

Gene 803

Gene 804

Gene 805

Gene 806

Gene 807

Gene 808

Gene 809

Gene 810

Gene 811

Gene 812

Gene 813

Gene 814

Gene 815

Gene 816

Gene 817

Gene 818

Gene 819

Gene 820

Gene 821

Gene 822

Gene 823

Gene 824

Gene 825

Gene 826

Gene 827

Gene 828

Gene 829

Gene 830

Gene 831

Gene 832

Gene 833

Gene 834

Gene 835

Gene 836

Gene 837

Gene 838

Gene 839

Gene 840

Gene 841

Gene 842

Gene 843

Gene 844

Gene 845

Gene 846

Gene 847

Gene 848

Gene 849

Gene 850

Gene 851

Gene 852

Gene 853

Gene 854

Gene 855

Gene 856

Gene 857

Gene 858

Gene 859

Gene 860

Gene 861

Gene 862

Gene 863

Gene 864

Gene 865

Gene 866

Gene 867

Gene 868

Gene 869

Gene 870

Gene 871

Gene 872

Gene 873

Gene 874

Gene 875

Gene 876

Gene 877

Gene 878

Gene 879

Gene 880

Gene 881

Gene 882

Gene 883

Gene 884

Gene 885

Gene 886

Gene 887

Gene 888

Gene 889

Gene 890

Gene 891

Gene 892

Gene 893

Gene 894

Gene 895

Gene 896

Gene 897

Gene 898

Gene 899

Gene 900

Gene 901

Gene 902

Gene 903

Gene 904

Gene 905

Gene 906

Gene 907

Gene 908

Gene 909

Gene 910

Gene 911

Gene 912

Gene 913

Gene 914

Gene 915

Gene 916

Gene 917

Gene 918

Gene 919

Gene 920

Gene 921

Gene 922

Gene 923

Gene 924

Gene 925

Gene 926

Gene 927

Gene 928

Gene 929

Gene 930

Gene 931

Gene 932

Gene 933

Gene 934

Gene 935

Gene 936

Gene 937

Gene 938

Gene 939

Gene 940

Gene 941

Gene 942

Gene 943

Gene 944

Gene 945

Gene 946

Gene 947

Gene 948

Gene 949

Gene 950

Gene 951

Gene 952

Gene 953

Gene 954

Gene 955

Gene 956

Gene 957

Gene 958

Gene 959

Gene 960

Gene 961

Gene 962

Gene 963

Gene 964

Gene 965

Gene 966

Gene 967

Gene 968

Gene 969

Gene 970

Gene 971

Gene 972

Gene 973

Gene 974

Gene 975

Gene 976

Gene 977

Gene 978

Gene 979

Gene 980

Gene 981

Gene 982

Gene 983

Gene 984

Gene 985

Gene 986

Gene 987

Gene 988

Gene 989

Gene 990

Gene 991

Gene 992

Gene 993

Gene 994

Gene 995

Gene 996

Gene 997

Gene 998

Gene 999

Gene 1000

Date: / /
 Page:

```

    fitness = fitness(population)
    selection(population)
    phenotype = gene_expression(population)
    best_fit = fitness(population)
    output = [p, best_fit, phenotype]
    return output

    IP + Specific field

    Input: Image processing output
    Gen 0: Best fitness = 915215.172, Fpr = [Exp(0.1), 0.1, 0.1]
    Gen 1: Best fitness = 981624.5547, Fpr = [Exp(0.1), 0.1, 0.1]
    :
    :
    Gen 50: Best fitness = 9826122.5704, Fpr = [Exp(0.1), 0.1, 0.1]

    Best expression found: [Exp(0.1), 0.1, 0.1]

    Parameters values from expression: [c, r, s, g]

    # Tolence
    The output from GEP study samples and
    with a few key differences being used instead
    directly on the values of the image processing
    but GEP links the expressions independent of its
    values, thus able to accommodate and more
    complex data sets.

    (population)
  
```

Date: / /
 Page:

Specific field:-

Predicting compressive strength of a material based on input variables

Output:-

Gen 0: Best expression $(C + (C * \sin(r)) - \sqrt{g(r)}), \text{MSE} = 12$

Gen 1: Best expression $(F + (F * \sin(r)) - \sqrt{g(r)}), \text{MSE} = 12$

Gen 2: Best expression $(W + (C * \sin(F)) - \sqrt{g(F)}), \text{MSE} = 12$

...

Gen 50: Best expression $(C + (W * \sin(F)) - \sqrt{g(F)}), \text{MSE} = 12$

Final Best Structural expression

$(C + (W * \sin(F)) - \sqrt{g(F)})$

Final MSE: 12

Desc:- Generating a. Formula for compressive strength through various input variable and for from a predicted strength to obtain a optimized solution

Statistics

Code:

```

import random
import numpy as np
from PIL import Image, ImageEnhance
from skimage.metrics import structural_similarity as ssim
from skimage.filters import sobel
from skimage import img_as_float
import math

# ----- Load Image -----
original = Image.open("input.jpg").convert("RGB")
original_np = np.array(original)

try:
    target = Image.open("target.jpg").convert("RGB")
    target_np = np.array(target)
    USE_REFERENCE = True
except:
    USE_REFERENCE = False

# ----- Function Set -----
def protected_div(x, y):
    try:
        return x / y if abs(y) > 1e-6 else 1.0
    
```

```

except:
    return 1.0

FUNCTIONS = [
    (lambda x: x, 'x'),
    (np.sin, 'sin'),
    (np.cos, 'cos'),
    (np.tan, 'tan'),
    (np.exp, 'exp'),
    (np.log1p, 'log1p'),
    (lambda x: x**2, 'square'),
    (np.sqrt, 'sqrt'),
    (np.abs, 'abs'),
]

# ----- Generate Random Expression Trees -----
def random_expr(depth=2):
    if depth == 0 or random.random() < 0.3:
        return 'x'
    func = random.choice(FUNCTIONS)[1]
    sub = random_expr(depth - 1)
    return f'{func}({sub})'

def evaluate_expr(expr_str, x_val):
    try:
        x = x_val
        return eval(expr_str, {"x": x, "sin": np.sin, "cos": np.cos, "tan": np.tan,
                                "exp": np.exp, "log1p": np.log1p, "sqrt": np.sqrt,
                                "abs": np.abs, "square": lambda x: x**2})
    except Exception as e:
        return 1.0

# ----- Individual is 3 Expressions -----
def init_population(size):
    return [[random_expr(2) for _ in range(3)] for _ in range(size)]

# ----- Apply Parameters -----
def apply_adjustments_from_expr(exprs):
    x = np.mean(original_np) / 255.0
    b = np.clip(evaluate_expr(exprs[0], x), 0.5, 2.0)
    c = np.clip(evaluate_expr(exprs[1], x), 0.5, 2.0)
    s = np.clip(evaluate_expr(exprs[2], x), 0.5, 2.0)

    img = original.copy()
    img = ImageEnhance.Brightness(img).enhance(b)
    img = ImageEnhance.Contrast(img).enhance(c)
    img = ImageEnhance.Sharpness(img).enhance(s)
    return img, [b, c, s]

# ----- Fitness -----
def fitness(ind):
    img, = apply_adjustments_from_expr(ind)
    img_np = np.array(img)

    if USE_REFERENCE:
        return ssim(target_np, img_np, channel_axis=2)
    else:

```

```

    gray = img.convert("L")
    gray_np = img_as_float(np.array(gray))
    entropy = -np.sum(gray_np * np.log2(gray_np + 1e-10))
    edge_strength = np.mean(sobel(gray_np))
    return entropy + edge_strength

# ----- Selection, Crossover, Mutation -----
def select(pop, fitnesses):
    i, j = random.sample(range(len(pop)), 2)
    return pop[i] if fitnesses[i] > fitnesses[j] else pop[j]

def crossover(p1, p2):
    child = []
    for a, b in zip(p1, p2):
        if random.random() < 0.5:
            child.append(a)
        else:
            child.append(b)
    return child

def mutate(expr):
    expr_list = expr.split()
    if random.random() < 0.3:
        return random_expr(2)
    return expr

def mutate_ind(ind):
    return [mutate(expr) if random.random() < 0.3 else expr for expr in ind]

# ----- Run GEP -----
def run_gep():
    POP_SIZE = 20
    N_GEN = 15
    ELITE = 2

    pop = init_population(POP_SIZE)

    for gen in range(N_GEN):
        fitnesses = [fitness(ind) for ind in pop]
        ranked = sorted(zip(pop, fitnesses), key=lambda x: x[1], reverse=True)
        best_ind, best_fit = ranked[0]

        print(f'Gen {gen}: Best fitness = {best_fit:.4f}, Exprs = {best_ind}')

        new_pop = [ind.copy() for ind, _ in ranked[:ELITE]]

        while len(new_pop) < POP_SIZE:
            p1, p2 = select(pop, fitnesses), select(pop, fitnesses)
            child = crossover(p1, p2)
            child = mutate_ind(child)
            new_pop.append(child)

        pop = new_pop

    best_img, best_params = apply_adjustments_from_expr(best_ind)
    best_img.save("gep_optimized.jpg")
    print("Best expressions found:", best_ind)

```

```
print("Parameter values from expressions:", best_params)  
display(best_img) # for Jupyter
```

```
# Run it  
run_gep()
```

Output:

```
Gen 0: Best fitness = 9153466.1126, Exprs = ['exp(sin(x))', 'abs(x)', 'cos(x(x))']  
Gen 1: Best fitness = 9153466.1126, Exprs = ['exp(sin(x))', 'abs(x)', 'cos(x(x))']  
Gen 2: Best fitness = 9153466.1126, Exprs = ['exp(sin(x))', 'abs(x)', 'cos(x(x))']  
Gen 3: Best fitness = 9153466.1126, Exprs = ['exp(sin(x))', 'abs(x)', 'cos(x(x))']  
Gen 4: Best fitness = 9153466.1126, Exprs = ['exp(sin(x))', 'abs(x)', 'cos(x(x))']  
Gen 5: Best fitness = 9816434.5797, Exprs = ['exp(cos(x))', 'x', 'cos(x(x))']  
Gen 6: Best fitness = 9816434.5797, Exprs = ['exp(cos(x))', 'x', 'cos(x(x))']  
Gen 7: Best fitness = 9816434.5797, Exprs = ['exp(cos(x))', 'x', 'cos(x(x))']  
Gen 8: Best fitness = 9816434.5797, Exprs = ['exp(cos(x))', 'x', 'cos(x(x))']  
Gen 9: Best fitness = 9826122.5704, Exprs = ['exp(cos(x))', 'x', 'sin(x)']  
Gen 10: Best fitness = 9826122.5704, Exprs = ['exp(cos(x))', 'x', 'sin(x)']  
Gen 11: Best fitness = 9826122.5704, Exprs = ['exp(cos(x))', 'x', 'sin(x)']  
Gen 12: Best fitness = 9826122.5704, Exprs = ['exp(cos(x))', 'x', 'sin(x)']  
Gen 13: Best fitness = 9826122.5704, Exprs = ['exp(cos(x))', 'x', 'sin(x)']  
Gen 14: Best fitness = 9826122.5704, Exprs = ['exp(cos(x))', 'x', 'sin(x)']  
Best expressions found: ['exp(cos(x))', 'x', 'sin(x)']  
Parameter values from expressions: [2.0, 0.5, 0.5]
```

Code:

```
import random  
import numpy as np  
import math  
  
# ----- DATASET -----  
dataset = np.array([  
    [350, 100, 180, 35],  
    [300, 120, 150, 28],  
    [400, 90, 200, 42],  
    [380, 110, 190, 38],  
    [360, 105, 180, 36],  
)  
  
variables = ['C', 'F', 'W']  
functions = ['+', '-', '*', '/', 'sin', 'cos', 'exp', 'log', 'sqrt']  
  
POP_SIZE = 50  
GENS = 50  
MUTATION_RATE = 0.2  
HEAD_LENGTH = 5  
TAIL_LENGTH = HEAD_LENGTH * (len(functions)-1) + 1  
NUM_GENES = 3 # More genes for structured combination
```

```

# ----- HELPER FUNCTIONS -----
def generate_gene():
    head = [random.choice(functions + variables) for _ in range(HEAD_LENGTH)]
    tail = [random.choice(variables) for _ in range(TAIL_LENGTH)]
    return head + tail

def gene_to_expression(gene):
    stack = []
    for symbol in reversed(gene):
        if symbol in variables:
            stack.append(symbol)
        elif symbol in functions:
            if symbol in ['sin', 'cos', 'exp', 'log', 'sqrt']:
                if stack:
                    a = stack.pop()
                    stack.append(f'{symbol}({a})')
            else: # binary operator
                if len(stack) >= 2:
                    a = stack.pop()
                    b = stack.pop()
                    stack.append(f'({a} {symbol} {b})')
            else:
                stack.append(random.choice(variables))
    return stack[0] if stack else random.choice(variables)

def safe_eval(expr, C, F, W):
    try:
        expr = expr.replace('/0', '/1e-6')
        expr = expr.replace('log(', 'np.log(abs(')
        expr = expr.replace('sqrt(', 'np.sqrt(abs(')
        return eval(expr, {"C": C, "F": F, "W": W, "np": np, "math": math})
    except:
        return np.inf

def generate_individual():
    return [generate_gene() for _ in range(NUM_GENES)]

# ----- Structured Multi-Gene Combination -----
def individual_to_expression(individual):
    # Start: gene1
    start = gene_to_expression(individual[0])
    # Middle: combine gene2 and gene3 nonlinearly
    middle = f'({gene_to_expression(individual[1])} * sin({gene_to_expression(individual[2])}))'
    # End: some nonlinear transformation of last gene
    end = f'- sqrt({gene_to_expression(individual[-1])})'
    # Full structured expression
    return f'({start} + {middle} {end})'

def fitness(individual):

```

```

    expr = individual_to_expression(individual)
    C_vals, F_vals, W_vals = dataset[:, 0], dataset[:, 1], dataset[:, 2]
    y_true = dataset[:, 3]
    y_pred = np.array([safe_eval(expr, C_vals[i], F_vals[i], W_vals[i]) for i in range(len(C_vals))])
    return np.mean((y_true - y_pred) ** 2)

# ----- GEP OPERATORS -----
def mutate(individual):
    ind = [g[:] for g in individual]
    if random.random() < MUTATION_RATE:
        gene_idx = random.randint(0, NUM_GENES-1)
        gene = ind[gene_idx]
        idx = random.randint(0, len(gene)-1)
        if idx < HEAD_LENGTH:
            gene[idx] = random.choice(functions + variables)
        else:
            gene[idx] = random.choice(variables)
        ind[gene_idx] = gene
    return ind

def crossover(parent1, parent2):
    gene_idx = random.randint(0, NUM_GENES-1)
    pt = random.randint(1, len(parent1[gene_idx])-1)
    child1_gene = parent1[gene_idx][:pt] + parent2[gene_idx][pt:]
    child2_gene = parent2[gene_idx][:pt] + parent1[gene_idx][pt:]
    child1, child2 = parent1[:], parent2[:]
    child1[gene_idx], child2[gene_idx] = child1_gene, child2_gene
    return child1, child2

def select(population, k=3):
    return min(random.sample(population, k), key=fitness)

# ----- MAIN GEP LOOP -----
def main_gep():
    population = [generate_individual() for _ in range(POP_SIZE)]

    for gen in range(GENS):
        new_population = []
        while len(new_population) < POP_SIZE:
            parent1 = select(population)
            parent2 = select(population)
            child1, child2 = crossover(parent1, parent2)
            child1 = mutate(child1)
            child2 = mutate(child2)
            new_population.extend([child1, child2])
        population = new_population[:POP_SIZE]

    best = min(population, key=fitness)
    best_expr = individual_to_expression(best)

```

```

best_fit = fitness(best)
print(f'Gen {gen}: Best expression: {best_expr}, MSE: {best_fit:.4f}')

final_best = min(population, key=fitness)
print("\nFinal Best Structured Expression (Compressive Strength in MPa):")
print(individual_to_expression(final_best))
print("Final MSE:", fitness(final_best))

if __name__ == "__main__":
    main_gep()

```

Output:

Gen 0: Best expression: $(C + (C * \sin(C)) - \sqrt{C})$, MSE: inf
 Gen 1: Best expression: $(F + (F * \sin(C)) - \sqrt{C})$, MSE: inf
 Gen 2: Best expression: $(W + (C * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 3: Best expression: $(F + (W * \sin(C)) - \sqrt{C})$, MSE: inf
 Gen 4: Best expression: $(F + (W * \sin(C)) - \sqrt{C})$, MSE: inf
 Gen 5: Best expression: $(F + (W * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 6: Best expression: $(W + (W * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 7: Best expression: $(W + (W * \sin(C)) - \sqrt{C})$, MSE: inf
 Gen 8: Best expression: $(C + (C * \sin(C)) - \sqrt{C})$, MSE: inf
 Gen 9: Best expression: $(W + (W * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 10: Best expression: $(C + (W * \sin(W)) - \sqrt{W})$, MSE: inf
 Gen 11: Best expression: $(C + (C * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 12: Best expression: $(C + (W * \sin(W)) - \sqrt{W})$, MSE: inf
 Gen 13: Best expression: $(C + (W * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 14: Best expression: $(C + (W * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 15: Best expression: $(W + (W * \sin(C)) - \sqrt{C})$, MSE: inf
 Gen 16: Best expression: $(F + (W * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 17: Best expression: $(F + (F * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 18: Best expression: $(W + (W * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 19: Best expression: $(W + (C * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 20: Best expression: $(C + (W * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 21: Best expression: $(C + (W * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 22: Best expression: $(C + (W * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 23: Best expression: $(W + (F * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 24: Best expression: $(W + (W * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 25: Best expression: $(C + (F * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 26: Best expression: $(W + (C * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 27: Best expression: $(C + (W * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 28: Best expression: $(C + (W * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 29: Best expression: $(F + (W * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 30: Best expression: $(C + (F * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 31: Best expression: $(W + (C * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 32: Best expression: $(F + (W * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 33: Best expression: $(C + (F * \sin(F)) - \sqrt{F})$, MSE: inf
 Gen 34: Best expression: $(W + (C * \sin(F)) - \sqrt{F})$, MSE: inf

Gen 35: Best expression: $(W + (W * \sin(F)) - \sqrt{F})$, MSE: inf
Gen 36: Best expression: $(C + (C * \sin(F)) - \sqrt{F})$, MSE: inf
Gen 37: Best expression: $(F + (F * \sin(F)) - \sqrt{F})$, MSE: inf
Gen 38: Best expression: $(W + (W * \sin(F)) - \sqrt{F})$, MSE: inf
Gen 39: Best expression: $(F + (C * \sin(F)) - \sqrt{F})$, MSE: inf
Gen 40: Best expression: $(W + (C * \sin(F)) - \sqrt{F})$, MSE: inf
Gen 41: Best expression: $(W + (C * \sin(F)) - \sqrt{F})$, MSE: inf
Gen 42: Best expression: $(W + (C * \sin(F)) - \sqrt{F})$, MSE: inf
Gen 43: Best expression: $(C + (F * \sin(F)) - \sqrt{F})$, MSE: inf
Gen 44: Best expression: $(W + (F * \sin(F)) - \sqrt{F})$, MSE: inf
Gen 45: Best expression: $(W + (C * \sin(F)) - \sqrt{F})$, MSE: inf
Gen 46: Best expression: $(W + (W * \sin(F)) - \sqrt{F})$, MSE: inf
Gen 47: Best expression: $(W + (C * \sin(F)) - \sqrt{F})$, MSE: inf
Gen 48: Best expression: $(C + (C * \sin(F)) - \sqrt{F})$, MSE: inf
Gen 49: Best expression: $(C + (W * \sin(F)) - \sqrt{F})$, MSE: inf

Final Best Structured Expression (Compressive Strength in MPa):

$(C + (W * \sin(F)) - \sqrt{F})$

Final MSE: inf

PROGRAM 3: PARTICLE SWARM OPTIMIZATION FOR FUNCTION OPTIMIZATION

Particle Swarm Optimization (PSO) is inspired by the social behavior of birds flocking or fish schooling. PSO is used to find optimal solutions by iteratively improving a candidate solution with regard to a given measure of quality. Implement the PSO algorithm using Python to optimize a mathematical function.

Algorithm:

DATE: 11/09/23
Page: 6

PARTICLE SWARM OPTIMIZATION
FOR FUNCTION OPTIMIZATION

Algorithm / Pseudo code

PSO (Function f , Dimension D , Particles N , Iteration N , p_{best} , Bounds, w , $c1$, $c2$, minimize = True)

// step 1
For $i = 1$ to N do
 $position[i] \leftarrow$ Random value within bounds
 $velocity[i] \leftarrow$ Random values
 $p_{best}[i] \leftarrow position[i]$
 $p_{best}[fitness[i]] \leftarrow fitness[i]$
End For

// step 2
 $g_{best} \leftarrow best$ p_{best} , Sort($fitness$)[1]
 $g_{best}[fitness] \leftarrow p(g_{best})$

// step 3
For $t = 1$ to T do
 For $i = 1$ to N do
 $fitness \leftarrow f(position[i])$
 if (minimize AND $fitness < p_{best}[fitness[i]]$) OR
 (NOT minimize AND $fitness > p_{best}[fitness[i]]$)
 then
 $p_{best}[i] \leftarrow position[i]$
 $p_{best}[fitness[i]] \leftarrow fitness$
 End if
 if (minimize AND $fitness < p(g_{best}[fitness])$) OR
 (NOT minimize AND $fitness > g_{best}[fitness]$)
 then
 $g_{best} \leftarrow position[i]$
 $g_{best}[fitness] \leftarrow fitness$
 End if
 End For

DATE: 11/09/23
Page: 7

For $j = 1$ to N do
 For $d = 1$ to D do
 $r1 \leftarrow random(0, 1)$
 $r2 \leftarrow random(0, 1)$
 $velocity[j][d] \leftarrow w * velocity[j][d] +$
 $c1 * r1 * (p_{best}[j][d] - position[j][d]) +$
 $c2 * r2 * (g_{best}[d] - position[j][d])$
 $position[j][d] \leftarrow position[j][d] + velocity[j][d]$
 $position[j][d] \leftarrow clamp(position[j][d], Bounds[d].min, Bounds[d].max)$
 End For
End For

Return g_{best} , $g_{best}[fitness]$

- Define g_{best} , p_{best} , $fitness$ fun.
- Evolve in the field of Energy optimization.

→ g_{best} :- Global best know position. It is the best among position of all the particles personal best. It guides all the other particles acting as the leader of the swarm.

While optimgⁿ → Energy optⁿ
PSO

Date: / /
Page: 7

pbest:- the best position/solution that a particle has individually found.
→ position in the search space.

pBest Fitness :- The fitness value at pBest[]

func fitness (position p[], math_function) {
// Input: position output: fitness
fitness ← 0.

for i = 1 to D do
 fitness ← fitness + math_function (position [i])

return fitness
}

Output:-
Optimizing the amount of solar PV panels, wind turbines & battery, based on percentage like consumption availability & load etc, cost etc.

Particles 25 Iteration 60

Iteration 10: Best = \$ 24498.91 Avg = \$ 26355.6 (conf 100%)
Iteration 20: Best = \$ 24364.16 Avg = \$ 25399.62 (conf 100%)
Iteration 30: Best = \$ 24566.69 Avg = \$ 24417.03 (conf 100%)
Iteration 60: Best = \$ 24366.19 Avg = \$ 24370.09 (conf 100%)

Result:
Optimal Configuration:
PV panels: 53 Energy output: 16.0 MW
Wind turbines: 0
Batteries: 0 Demand Met: Yes
Total cost: \$ 24,366.19

Code:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.patches import Rectangle
from matplotlib.animation import FuncAnimation
import pandas as pd
from typing import Tuple, Dict, List
import time

# Set style for better plots
plt.style.use('seaborn-v0_8')
sns.set_palette("husl")
```

```

class RenewableEnergyOptimizer:
    def __init__(self, load_demand=16, pv_cost=450, wind_cost=1_200_000, battery_cost=2500):
        """
        Enhanced Renewable Energy System Optimizer with visualization capabilities

        Parameters:
        - load_demand: Daily energy demand in MWh
        - pv_cost: Cost per PV panel ($)
        - wind_cost: Cost per wind turbine ($)
        - battery_cost: Cost per battery unit ($)
        """
        self.load_demand = load_demand
        self.pv_cost = pv_cost
        self.wind_cost = wind_cost
        self.battery_cost = battery_cost

        # Energy production rates (MWh/day)
        self.pv_energy_rate = 0.3
        self.wind_energy_rate = 5.0
        self.battery_energy_rate = 0.1

        # Operating costs ($/year)
        self.pv_op_cost = 5
        self.wind_op_cost = 200
        self.battery_op_cost = 20

        # Optimization history
        self.convergence_history = []
        self.particle_history = []

    def energy_cost(self, variables: np.ndarray, return_details=False) -> float:
        """
        Calculate total energy system cost with detailed breakdown
        """
        num_pv, num_wind, num_battery = variables

        # Capital costs
        cost_pv = self.pv_cost * num_pv
        cost_wind = self.wind_cost * num_wind
        cost_battery = self.battery_cost * num_battery

        # Operating costs (annual)
        op_pv = self.pv_op_cost * num_pv
        op_wind = self.wind_op_cost * num_wind
        op_battery = self.battery_op_cost * num_battery

        # Energy contribution (MWh/day)
        energy_pv = num_pv * self.pv_energy_rate
        energy_wind = num_wind * self.wind_energy_rate
        energy_battery = num_battery * self.battery_energy_rate
        total_energy = energy_pv + energy_wind + energy_battery

        # Penalty for not meeting demand
        penalty = 0
        if total_energy < self.load_demand:
            penalty = 1e6 * (self.load_demand - total_energy)

```

```

total_cost = cost_pv + cost_wind + cost_battery + op_pv + op_wind + op_battery + penalty

if return_details:
    return {
        'total_cost': total_cost,
        'capital_costs': {'pv': cost_pv, 'wind': cost_wind, 'battery': cost_battery},
        'operating_costs': {'pv': op_pv, 'wind': op_wind, 'battery': op_battery},
        'energy_production': {'pv': energy_pv, 'wind': energy_wind, 'battery': energy_battery},
        'total_energy': total_energy,
        'penalty': penalty,
        'demand_met': total_energy >= self.load_demand
    }

return total_cost

def pso_optimize(self, num_particles=20, max_iter=50, bounds=None, verbose=True):
    """
    Enhanced Particle Swarm Optimization with tracking
    """
    if bounds is None:
        bounds = [(0, 200), (0, 5), (0, 500)] # [pv_min_max, wind_min_max, battery_min_max]

    dim = 3
    w = 0.7 # inertia weight
    c1, c2 = 1.5, 1.5 # acceleration constants

    # Initialize particles
    pos = np.zeros((num_particles, dim))
    vel = np.zeros((num_particles, dim))

    for i in range(dim):
        pos[:, i] = np.random.uniform(bounds[i][0], bounds[i][1], num_particles)
        vel[:, i] = np.random.uniform(-1, 1, num_particles)

    # Initialize personal and global bests
    pbest = pos.copy()
    pbest_val = np.array([self.energy_cost(x) for x in pos])
    gbest = pbest[np.argmin(pbest_val)].copy()
    gbest_val = np.min(pbest_val)

    # Reset tracking arrays
    self.convergence_history = []
    self.particle_history = []

    print("Starting Particle Swarm Optimization...")
    print(f'Particles: {num_particles}, Iterations: {max_iter}')
    print("-" * 60)

    for iteration in range(max_iter):
        for i in range(num_particles):
            # Update velocity
            r1, r2 = np.random.rand(dim), np.random.rand(dim)
            vel[i] = w * vel[i] + c1 * r1 * (pbest[i] - pos[i]) + c2 * r2 * (gbest - pos[i])

            # Update position
            pos[i] += vel[i]

```

```

        # Apply bounds
        for j in range(dim):
            pos[i, j] = np.clip(pos[i, j], bounds[j][0], bounds[j][1])

        # Evaluate fitness
        fitness = self.energy_cost(pos[i])

        # Update personal best
        if fitness < pbest_val[i]:
            pbest[i] = pos[i].copy()
            pbest_val[i] = fitness

        # Update global best
        min_idx = np.argmin(pbest_val)
        if pbest_val[min_idx] < gbest_val:
            gbest = pbest[min_idx].copy()
            gbest_val = pbest_val[min_idx]

        # Store convergence data
        avg_fitness = np.mean(pbest_val)
        self.convergence_history.append({
            'iteration': iteration + 1,
            'best_cost': gbest_val,
            'avg_cost': avg_fitness,
            'std_cost': np.std(pbest_val)
        })

        # Store particle positions for animation
        if iteration % 5 == 0: # Store every 5th iteration to save memory
            self.particle_history.append(pos.copy())

        if verbose and (iteration + 1) % 10 == 0:
            print(f"Iteration {iteration + 1:3d}: Best = ${gbest_val:12,.2f}, "
                  f"Avg = ${avg_fitness:12,.2f}, "
                  f"Config = [{int(gbest[0])}, {int(gbest[1])}, {int(gbest[2])}]")

        print("-" * 60)
        print("Optimization Complete!")

        return gbest, gbest_val

def plot_convergence(self, figsize=(12, 8)):
    """
    Plot convergence history with multiple metrics
    """
    if not self.convergence_history:
        print("No convergence data available. Run optimization first.")
        return

    df = pd.DataFrame(self.convergence_history)

    fig, axes = plt.subplots(2, 2, figsize=figsize)
    fig.suptitle('PSO Convergence Analysis', fontsize=16, fontweight='bold')

    # Best cost convergence
    axes[0, 0].plot(df['iteration'], df['best_cost'], 'b-', linewidth=2, marker='o', markersize=4)

```

```

axes[0, 0].set_title('Best Cost Convergence')
axes[0, 0].set_xlabel('Iteration')
axes[0, 0].set_ylabel('Cost ($)')
axes[0, 0].grid(True, alpha=0.3)
axes[0, 0].ticklabel_format(style='scientific', axis='y', scilimits=(0,0))

# Average cost convergence
axes[0, 1].plot(df['iteration'], df['avg_cost'], 'r-', linewidth=2, marker='s', markersize=4)
axes[0, 1].fill_between(df['iteration'],
                        df['avg_cost'] - df['std_cost'],
                        df['avg_cost'] + df['std_cost'],
                        alpha=0.3, color='red')
axes[0, 1].set_title('Average Cost  $\pm$  Std Dev')
axes[0, 1].set_xlabel('Iteration')
axes[0, 1].set_ylabel('Cost ($)')
axes[0, 1].grid(True, alpha=0.3)
axes[0, 1].ticklabel_format(style='scientific', axis='y', scilimits=(0,0))

# Cost improvement rate
cost_improvement = np.diff(df['best_cost'])
axes[1, 0].plot(df['iteration'][1:], cost_improvement, 'g-', linewidth=2)
axes[1, 0].set_title('Cost Improvement Rate')
axes[1, 0].set_xlabel('Iteration')
axes[1, 0].set_ylabel('Cost Change ($)')
axes[1, 0].grid(True, alpha=0.3)
axes[1, 0].axhline(y=0, color='black', linestyle='--', alpha=0.5)

# Convergence rate (log scale)
axes[1, 1].semilogy(df['iteration'], df['best_cost'], 'purple', linewidth=2, marker='^', markersize=4)
axes[1, 1].set_title('Best Cost (Log Scale)')
axes[1, 1].set_xlabel('Iteration')
axes[1, 1].set_ylabel('Cost ($) - Log Scale')
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

def plot_cost_breakdown(self, solution, figsize=(14, 6)):
    """
    Visualize cost breakdown of the optimal solution
    """
    details = self.energy_cost(solution, return_details=True)

    fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=figsize)
    fig.suptitle('Optimal Solution Analysis', fontsize=16, fontweight='bold')

    # Cost breakdown pie chart
    capital_costs = list(details['capital_costs'].values())
    operating_costs = list(details['operating_costs'].values())

    all_costs = capital_costs + operating_costs
    labels = ['PV Capital', 'Wind Capital', 'Battery Capital',
              'PV Operating', 'Wind Operating', 'Battery Operating']

    # Filter out zero costs
    non_zero_costs = [(cost, label) for cost, label in zip(all_costs, labels) if cost > 0]
    if non_zero_costs:

```

```

costs, labels = zip(*non_zero_costs)
colors = plt.cm.Set3(np.linspace(0, 1, len(costs)))

wedges, texts, autotexts = ax1.pie(costs, labels=labels, autopct='%1.1f%%',
                                   colors=colors, startangle=90)
ax1.set_title('Cost Breakdown')

# Energy production bar chart
energy_sources = ['PV Panels', 'Wind Turbines', 'Batteries']
energy_values = list(details['energy_production'].values())
colors = ['gold', 'skyblue', 'lightgreen']

bars = ax2.bar(energy_sources, energy_values, color=colors, alpha=0.8)
ax2.set_title('Energy Production by Source')
ax2.set_ylabel('Energy (MWh/day)')
ax2.axhline(y=self.load_demand, color='red', linestyle='--',
            label=f'Demand ({self.load_demand} MWh)')
ax2.legend()

# Add value labels on bars
for bar, value in zip(bars, energy_values):
    height = bar.get_height()
    ax2.text(bar.get_x() + bar.get_width()/2., height + 0.1,
             f'{value:.1f}', ha='center', va='bottom')

# System configuration
config = [int(solution[0]), int(solution[1]), int(solution[2])]
components = ['PV Panels', 'Wind Turbines', 'Batteries']
colors = ['orange', 'cyan', 'lime']

bars = ax3.bar(components, config, color=colors, alpha=0.8)
ax3.set_title('System Configuration')
ax3.set_ylabel('Number of Units')

# Add value labels on bars
for bar, value in zip(bars, config):
    height = bar.get_height()
    ax3.text(bar.get_x() + bar.get_width()/2., height + 0.5,
             f'{value}', ha='center', va='bottom', fontweight='bold')

plt.tight_layout()
plt.show()

return details

def sensitivity_analysis(self, solution, parameter_ranges=None, figsize=(15, 10)):
    """
    Perform sensitivity analysis on key parameters
    """
    if parameter_ranges is None:
        parameter_ranges = {
            'load_demand': np.linspace(10, 25, 20),
            'pv_cost': np.linspace(300, 600, 20),
            'wind_cost': np.linspace(800_000, 1_600_000, 20),
            'battery_cost': np.linspace(1500, 3500, 20)
        }

```

```

fig, axes = plt.subplots(2, 2, figsize=figsize)
fig.suptitle('Sensitivity Analysis', fontsize=16, fontweight='bold')

original_values = {
    'load_demand': self.load_demand,
    'pv_cost': self.pv_cost,
    'wind_cost': self.wind_cost,
    'battery_cost': self.battery_cost
}

for idx, (param, values) in enumerate(parameter_ranges.items()):
    ax = axes[idx // 2, idx % 2]
    costs = []

    for value in values:
        # Temporarily change parameter
        setattr(self, param, value)
        cost = self.energy_cost(solution)
        costs.append(cost)

    # Restore original value
    setattr(self, param, original_values[param])

    ax.plot(values, costs, 'o-', linewidth=2, markersize=6)
    ax.set_title(f'Sensitivity to {param.replace("_", " ").title()}')
    ax.set_xlabel(param.replace("_", " ").title())
    ax.set_ylabel('Total Cost ($)')
    ax.grid(True, alpha=0.3)
    ax.ticklabel_format(style='scientific', axis='y', scilimits=(0,0))

    # Mark current value
    current_val = original_values[param]
    current_cost = self.energy_cost(solution)
    ax.axvline(x=current_val, color='red', linestyle='--', alpha=0.7)
    ax.plot(current_val, current_cost, 'ro', markersize=8,
            label=f'Current: {current_val:,.0f}')
    ax.legend()

plt.tight_layout()
plt.show()

def compare_scenarios(self, scenarios, figsize=(12, 8)):
    """
    Compare different demand scenarios
    """
    results = []

    for scenario_name, demand in scenarios.items():
        original_demand = self.load_demand
        self.load_demand = demand

        # Quick optimization for each scenario
        solution, cost = self.pso_optimize(num_particles=15, max_iter=30, verbose=False)
        details = self.energy_cost(solution, return_details=True)

        results.append({
            'scenario': scenario_name,

```

```

        'demand': demand,
        'solution': solution,
        'cost': cost,
        'pv_panels': int(solution[0]),
        'wind_turbines': int(solution[1]),
        'batteries': int(solution[2]),
        'total_energy': details['total_energy']
    })

    self.load_demand = original_demand # Restore

df = pd.DataFrame(results)

fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=figsize)
fig.suptitle('Scenario Comparison Analysis', fontsize=16, fontweight='bold')

# Cost comparison
bars = ax1.bar(df['scenario'], df['cost'], color='steelblue', alpha=0.7)
ax1.set_title('Total System Cost by Scenario')
ax1.set_ylabel('Cost ($)')
ax1.tick_params(axis='x', rotation=45)
ax1.ticklabel_format(style='scientific', axis='y', scilimits=(0,0))

# Add value labels
for bar, cost in zip(bars, df['cost']):
    height = bar.get_height()
    ax1.text(bar.get_x() + bar.get_width()/2., height,
             f'${cost:,.0f}', ha='center', va='bottom', fontsize=8)

# Component comparison
components = ['pv_panels', 'wind_turbines', 'batteries']
x = np.arange(len(df))
width = 0.25

for i, component in enumerate(components):
    ax2.bar(x + i*width, df[component], width,
            label=component.replace('_', ' ').title(),
            alpha=0.7)

ax2.set_title('System Configuration by Scenario')
ax2.set_xlabel('Scenario')
ax2.set_ylabel('Number of Units')
ax2.set_xticks(x + width)
ax2.set_xticklabels(df['scenario'], rotation=45)
ax2.legend()

# Demand vs Energy Production
ax3.scatter(df['demand'], df['total_energy'], s=100, alpha=0.7, color='green')
ax3.plot([df['demand'].min(), df['demand'].max()],
        [df['demand'].min(), df['demand'].max()],
        'r--', alpha=0.5, label='Perfect Match')
ax3.set_title('Energy Production vs Demand')
ax3.set_xlabel('Demand (MWh/day)')
ax3.set_ylabel('Production (MWh/day)')
ax3.legend()
ax3.grid(True, alpha=0.3)

```

```

# Cost efficiency (cost per MWh)
efficiency = df['cost'] / df['demand']
ax4.bar(df['scenario'], efficiency, color='coral', alpha=0.7)
ax4.set_title('Cost Efficiency ($/MWh of Demand)')
ax4.set_ylabel('Cost per MWh ($)')
ax4.tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()

return df

def main():
    """
    Main function demonstrating the enhanced optimization tool
    """
    print("=" * 70)
    print("ENHANCED RENEWABLE ENERGY SYSTEM OPTIMIZATION")
    print("=" * 70)

    # Initialize optimizer
    optimizer = RenewableEnergyOptimizer(
        load_demand=16,
        pv_cost=450,
        wind_cost=1_200_000,
        battery_cost=2500
    )

    # Run optimization
    print("\n1. Running Primary Optimization...")
    best_solution, best_cost = optimizer.pso_optimize(num_particles=25, max_iter=60)

    # Display results
    details = optimizer.energy_cost(best_solution, return_details=True)
    print(f"\nOPTIMAL CONFIGURATION:")
    print(f" PV Panels    : {int(best_solution[0]):,}")
    print(f" Wind Turbines : {int(best_solution[1]):,}")
    print(f" Batteries     : {int(best_solution[2]):,}")
    print(f" Total Cost    : ${best_cost:,.2f}")
    print(f" Energy Output : {details['total_energy']:.1f} MWh/day")
    print(f" Demand Met    : {'Yes' if details['demand_met'] else 'No'}")

    # Visualizations
    print("\n2. Generating Convergence Analysis...")
    optimizer.plot_convergence()

    print("\n3. Generating Cost Breakdown Analysis...")
    optimizer.plot_cost_breakdown(best_solution)

    print("\n4. Performing Sensitivity Analysis...")
    optimizer.sensitivity_analysis(best_solution)

    print("\n5. Comparing Different Demand Scenarios...")
    scenarios = {
        'Low Demand': 12,
        'Current': 16,
        'High Demand': 20,

```

```

        'Peak Demand': 25
    }
    scenario_results = optimizer.compare_scenarios(scenarios)

    print("\nScenario Comparison Results:")
    print(scenario_results[['scenario', 'demand', 'cost', 'pv_panels', 'wind_turbines', 'batteries']].to_string(index=False))

    print("\n" + "=" * 70)
    print("ANALYSIS COMPLETE - Check the generated plots for detailed insights!")
    print("=" * 70)

if __name__ == "__main__":
    main()

```

Ouput:

ENHANCED RENEWABLE ENERGY SYSTEM OPTIMIZATION

1. Running Primary Optimization...
 Starting Particle Swarm Optimization...
 Particles: 25, Iterations: 60

```

-----
Iteration 10: Best = $ 24,498.91, Avg = $ 28,355.16, Config = [53, 0, 0]
Iteration 20: Best = $ 24,304.16, Avg = $ 25,389.48, Config = [53, 0, 0]
Iteration 30: Best = $ 24,272.36, Avg = $ 24,908.85, Config = [53, 0, 0]
Iteration 40: Best = $ 24,266.69, Avg = $ 24,562.49, Config = [53, 0, 0]
Iteration 50: Best = $ 24,266.69, Avg = $ 24,417.03, Config = [53, 0, 0]
Iteration 60: Best = $ 24,266.69, Avg = $ 24,370.09, Config = [53, 0, 0]
-----

```

Optimization Complete!

OPTIMAL CONFIGURATION:

PV Panels : 53
 Wind Turbines : 0
 Batteries : 0
 Total Cost : \$24,266.69
 Energy Output : 16.0 MWh/day
 Demand Met : Yes

PROGRAM 4: ANT COLONY OPTIMIZATION FOR THE TRAVELING SALESMAN PROBLEM:

The foraging behavior of ants has inspired the development of optimization algorithms that can solve complex problems such as the Traveling Salesman Problem (TSP). Ant Colony Optimization (ACO) simulates the way ants find the shortest path between food sources and their nest. Implement the ACO algorithm using Python to solve the TSP, where the objective is to find the shortest possible route that visits a list of cities and returns to the origin city.

Algorithm:

The image shows a handwritten algorithm for the Ant Colony Optimization (ACO) algorithm applied to the Traveling Salesman Problem (TSP). The algorithm is written on two pages of a notebook, with the title 'ANT COLONY OPTIMIZATION FOR TRAVELING SALESMAN PROBLEM' at the top of the first page. The algorithm is written in a mix of English and mathematical notation, with some parts in Hindi. The first page includes the initialization of parameters (m, max_iter, alpha, beta, Q, P, T) and the main loop for finding the shortest path. The second page includes the update of pheromones and the calculation of the total cost. The algorithm is written in a clear, step-by-step manner, with comments in Hindi explaining the steps. The final output is the shortest path and the total cost.

```

Date: 09/10/25
Page: 1

ANT COLONY OPTIMIZATION FOR
TRAVELING SALESMAN PROBLEM

Algorithm/Pseudocode:-
ACO() :-
Step 1:-
Initialize parameters () :-
m ← no. of ants
max_iter ← no. of iterations
α ← pheromone importance (alpha)
β ← heuristic importance (beta)
Q ← constant
P ← 1/Q
T ← 0 for all edges (i,j)

Step 2:-
Function findShortestPath (edges (i,j), start, goal) :-
best_sol ← NULL
best_cost ← INF
for i = 1 to max_iter do:
    for ant = 1 to m do:
        path[k] = random(i,j)
        partial_solution ← S_k ← (start, node)
        while S_k != goal do:
            next_nodes ← all feasible neighbours
            for each next_node:
                P_ij ← [T_ij^α * η_ij^β] / [Σ T_ik^α * η_ik^β]
            next_node ← top(sorted(P_ij))
            position[k] ← next_node
            ADD S_k ← next_node
        END WHILE
    END FOR
    // compute total cost
    cost ← f(S_k)
    if cost < best_cost then
        best_sol ← S_k
        best_cost ← cost
    END IF
END FOR
return best_sol, best_cost

END
    
```

1/10/25

Date: 09/10/25
Page: 2

```

IF S_k < best_cost then
    best_sol ← S_k
    best_cost ← f(S_k)
END IF
END FOR

// Step 4:- Update pheromone
T_ij ← (1-ρ) * T_ij
for all (i,j)
    T_ij ← (1-ρ) * T_ij + (Q/P(S_k))
END FOR

// New pheromones
for k = 1 to m do
    for edges (i,j) in S_k do
        T_ij ← T_ij + (Q/P(S_k))
    END FOR
END FOR

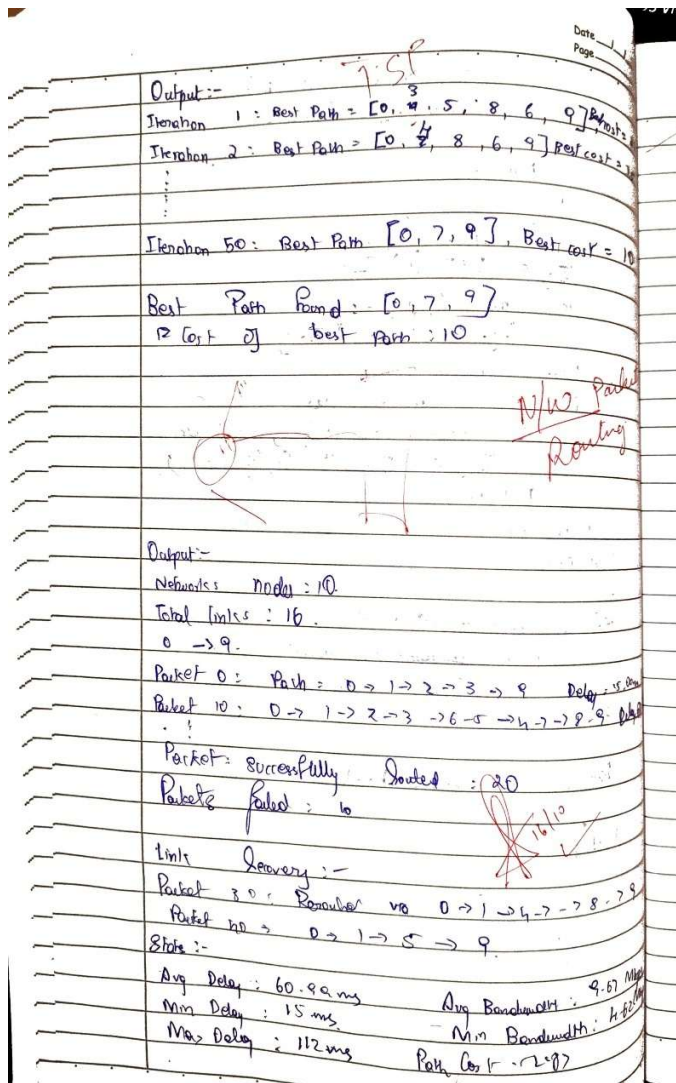
END FOR

// Function dist f(S_k)
// Return distance the travelled distance
// from start node to current node

Return best_sol, best_cost

END
    
```

Traffic Management



Code:

```
import random
import math
from collections import defaultdict, deque
import matplotlib.pyplot as plt
import networkx as nx
import time

class NetworkLink:
    """Represents a network link with dynamic properties"""
    def __init__(self, bandwidth, base_delay, packet_loss=0.0):
        self.bandwidth = bandwidth # Mbps
        self.base_delay = base_delay # ms
        self.current_load = 0.0 # Current traffic load
        self.packet_queue = deque()
        self.packet_loss = packet_loss
        self.failed = False
        self.congestion_history = []
```

```

def get_current_delay(self):
    """Calculate actual delay based on congestion"""
    if self.failed:
        return float('inf')

    # Delay increases exponentially with load
    utilization = self.current_load / self.bandwidth
    congestion_factor = 1 + (utilization ** 3) * 10 # Non-linear congestion
    return self.base_delay * congestion_factor

def get_available_bandwidth(self):
    """Get currently available bandwidth"""
    if self.failed:
        return 0.0
    return max(0, self.bandwidth - self.current_load)

def add_traffic(self, traffic_amount):
    """Add traffic to the link"""
    self.current_load = min(self.bandwidth * 1.2, self.current_load + traffic_amount)

def decay_traffic(self, decay_rate=0.1):
    """Simulate traffic decay over time"""
    self.current_load = max(0, self.current_load * (1 - decay_rate))
    self.congestion_history.append(self.current_load / self.bandwidth)
    if len(self.congestion_history) > 100:
        self.congestion_history.pop(0)

class DataPacket:
    """Represents an actual data packet with requirements"""
    def __init__(self, packet_id, source, destination, size, priority=1, bandwidth_req=0):
        self.packet_id = packet_id
        self.source = source
        self.destination = destination
        self.size = size # MB
        self.priority = priority # 1-5, higher = more important
        self.bandwidth_req = bandwidth_req # Minimum bandwidth required (Mbps)
        self.created_time = time.time()
        self.delivery_time = None
        self.path_taken = []
        self.hops = 0

class NetworkNode:
    """Represents a router with queuing and processing"""
    def __init__(self, node_id, processing_capacity=100):
        self.node_id = node_id
        self.pheromone_table = defaultdict(lambda: defaultdict(float))
        self.routing_table = defaultdict(lambda: defaultdict(float))
        self.neighbors = []
        self.packet_queue = deque()
        self.processing_capacity = processing_capacity # packets/sec
        self.packets_processed = 0
        self.packets_dropped = 0

    def initialize_pheromones(self, destination, neighbors, initial_value=0.1):
        """Initialize pheromone values for a destination"""
        for neighbor in neighbors:

```

```

        self.pheromone_table[destination][neighbor] = initial_value
        self.routing_table[destination][neighbor] = 1.0 / len(neighbors)

def get_queue_length(self):
    """Get current queue occupancy"""
    return len(self.packet_queue)

class NetworkGraph:
    """Represents the network topology with dynamic properties"""
    def __init__(self):
        self.nodes = {}
        self.links = {}
        self.traffic_matrix = defaultdict(lambda: defaultdict(float))
        self.active_flows = []

    def add_node(self, node_id, processing_capacity=100):
        """Add a node to the network"""
        if node_id not in self.nodes:
            self.nodes[node_id] = NetworkNode(node_id, processing_capacity)

    def add_link(self, node1, node2, bandwidth, delay, packet_loss=0.0):
        """Add a bidirectional link with capacity and delay"""
        self.add_node(node1)
        self.add_node(node2)

        self.links[(node1, node2)] = NetworkLink(bandwidth, delay, packet_loss)
        self.links[(node2, node1)] = NetworkLink(bandwidth, delay, packet_loss)

        if node2 not in self.nodes[node1].neighbors:
            self.nodes[node1].neighbors.append(node2)
        if node1 not in self.nodes[node2].neighbors:
            self.nodes[node2].neighbors.append(node1)

    def get_link(self, node1, node2):
        """Get link between two nodes"""
        return self.links.get((node1, node2))

    def simulate_link_failure(self, node1, node2):
        """Simulate a link failure"""
        if (node1, node2) in self.links:
            self.links[(node1, node2)].failed = True
            self.links[(node2, node1)].failed = True

    def restore_link(self, node1, node2):
        """Restore a failed link"""
        if (node1, node2) in self.links:
            self.links[(node1, node2)].failed = False
            self.links[(node2, node1)].failed = False

    def update_link_loads(self, decay_rate=0.1):
        """Update all link loads (simulate traffic decay)"""
        for link in self.links.values():
            link.decay_traffic(decay_rate)

class RoutingAnt:
    """Ant that explores routes while considering network state"""
    def __init__(self, source, destination, packet_size=1.0, bandwidth_req=0, priority=1):

```

```

self.source = source
self.destination = destination
self.packet_size = packet_size
self.bandwidth_req = bandwidth_req
self.priority = priority
self.path = [source]
self.total_delay = 0.0
self.total_cost = 0.0
self.visited = {source}
self.bottleneck_bandwidth = float('inf')
self.feasible = True

def calculate_link_cost(self, link, node_queue_length):
    """Calculate comprehensive cost for a link"""
    if link.failed:
        return float('inf')

    # Multiple factors influence cost
    delay_cost = link.get_current_delay()

    # Bandwidth availability factor
    available_bw = link.get_available_bandwidth()
    if available_bw < self.bandwidth_req:
        self.feasible = False
        return float('inf')

    bw_factor = 1.0 / (1.0 + available_bw) if available_bw > 0 else 10.0

    # Queue length factor (congestion at node)
    queue_factor = 1.0 + (node_queue_length * 0.1)

    # Packet loss factor
    loss_factor = 1.0 + (link.packet_loss * 5.0)

    # Combined cost with priority consideration
    cost = (delay_cost * bw_factor * queue_factor * loss_factor) / self.priority

    return cost

def select_next_node(self, current_node, network, alpha=1.0, beta=2.0):
    """Select next node based on multiple QoS factors"""
    node = network.nodes[current_node]
    neighbors = node.neighbors

    # Filter out visited neighbors and failed links
    available_neighbors = []
    for n in neighbors:
        if n not in self.visited:
            link = network.get_link(current_node, n)
            if link and not link.failed:
                available_neighbors.append(n)

    if not available_neighbors:
        return None

    # Calculate probabilities based on pheromone and link quality
    probabilities = []

```

```

costs = []

for neighbor in available_neighbors:
    # Pheromone level
    tau = node.pheromone_table[self.destination].get(neighbor, 0.1)

    # Link quality heuristic (inverse of cost)
    link = network.get_link(current_node, neighbor)
    next_node_obj = network.nodes[neighbor]
    cost = self.calculate_link_cost(link, next_node_obj.get_queue_length())
    costs.append(cost)

    if cost == float('inf'):
        probabilities.append(0)
    else:
        eta = 1.0 / cost
        # Combined probability
        prob = (tau ** alpha) * (eta ** beta)
        probabilities.append(prob)

# Check if any feasible path exists
if all(p == 0 for p in probabilities):
    return None

# Normalize probabilities
total = sum(probabilities)
if total > 0:
    probabilities = [p / total for p in probabilities]
else:
    return None

# Select next node
next_node = random.choices(available_neighbors, weights=probabilities)[0]
return next_node

def move_to(self, next_node, network):
    """Move ant to next node and update metrics"""
    current_node = self.path[-1]
    link = network.get_link(current_node, next_node)

    delay = link.get_current_delay()
    self.total_delay += delay

    # Track bottleneck bandwidth
    available_bw = link.get_available_bandwidth()
    self.bottleneck_bandwidth = min(self.bottleneck_bandwidth, available_bw)

    # Calculate comprehensive cost
    node = network.nodes[next_node]
    self.total_cost += self.calculate_link_cost(link, node.get_queue_length())

    self.path.append(next_node)
    self.visited.add(next_node)

class ACOPacketRouter:
    """Advanced ACO router with QoS awareness"""
    def __init__(self, network, alpha=1.0, beta=2.5, evaporation=0.15,

```

```

        q_constant=100, num_ants=15):
    self.network = network
    self.alpha = alpha
    self.beta = beta
    self.evaporation = evaporation
    self.q_constant = q_constant
    self.num_ants = num_ants
    self.initialize_pheromones()
    self.best_paths = {}
    self.path_metrics = defaultdict(list)
    self.packets_routed = 0
    self.packets_failed = 0

def initialize_pheromones(self):
    """Initialize pheromone tables"""
    for node_id, node in self.network.nodes.items():
        for dest in self.network.nodes.keys():
            if dest != node_id:
                node.initialize_pheromones(dest, node.neighbors)

def send_ants(self, source, destination, packet_size=1.0, bandwidth_req=0,
              priority=1, num_ants=None):
    """Send ants to explore routes"""
    if num_ants is None:
        num_ants = self.num_ants

    successful_ants = []

    for _ in range(num_ants):
        ant = RoutingAnt(source, destination, packet_size, bandwidth_req, priority)
        current = source

        # Forward ant movement
        max_hops = len(self.network.nodes) * 2
        while current != destination and len(ant.path) < max_hops:
            next_node = ant.select_next_node(current, self.network,
                                             self.alpha, self.beta)
            if next_node is None:
                break

            ant.move_to(next_node, self.network)
            current = next_node

        # Check if ant reached destination successfully
        if current == destination and ant.feasible:
            successful_ants.append(ant)

    return successful_ants

def update_pheromones(self, ants):
    """Update pheromones with quality-based deposition"""
    # Evaporation
    for node in self.network.nodes.values():
        for dest in node.pheromone_table:
            for neighbor in node.pheromone_table[dest]:
                node.pheromone_table[dest][neighbor] *= (1 - self.evaporation)

```

```

# Deposit pheromones based on path quality
for ant in ants:
    # Better paths (lower cost, lower delay) get more pheromone
    quality_metric = self.q_constant / (ant.total_cost + 1.0)

    # Bonus for paths with good bandwidth
    if ant.bottleneck_bandwidth > 0:
        bw_bonus = min(2.0, ant.bottleneck_bandwidth / 10.0)
        quality_metric *= bw_bonus

    # Priority bonus
    quality_metric *= (ant.priority / 3.0)

    # Deposit along path
    for i in range(len(ant.path) - 1):
        current = ant.path[i]
        next_node = ant.path[i + 1]

        node = self.network.nodes[current]
        node.pheromone_table[ant.destination][next_node] += quality_metric

def update_routing_tables(self):
    """Update routing probability tables"""
    for node in self.network.nodes.values():
        for dest in node.pheromone_table:
            neighbors = list(node.pheromone_table[dest].keys())
            if not neighbors:
                continue

            total_pheromone = sum(node.pheromone_table[dest].values())

            if total_pheromone > 0:
                for neighbor in neighbors:
                    node.routing_table[dest][neighbor] = \
                        node.pheromone_table[dest][neighbor] / total_pheromone

def route_packet(self, packet):
    """Route a data packet using current pheromone trails"""
    ants = self.send_ants(packet.source, packet.destination,
                           packet.size, packet.bandwidth_req, packet.priority)

    if ants:
        self.update_pheromones(ants)
        self.update_routing_tables()

    # Select best path considering multiple factors
    best_ant = min(ants, key=lambda a: a.total_cost)

    # Update link loads along chosen path
    for i in range(len(best_ant.path) - 1):
        link = self.network.get_link(best_ant.path[i], best_ant.path[i + 1])
        link.add_traffic(packet.size)

    # Store path info
    key = (packet.source, packet.destination)
    self.best_paths[key] = (best_ant.path, best_ant.total_delay,
                           best_ant.bottleneck_bandwidth)

```

```

        self.path_metrics[key].append({
            'delay': best_ant.total_delay,
            'bandwidth': best_ant.bottleneck_bandwidth,
            'cost': best_ant.total_cost
        })

        packet.path_taken = best_ant.path
        packet.hops = len(best_ant.path) - 1
        packet.delivery_time = best_ant.total_delay

        self.packets_routed += 1
        return best_ant.path, best_ant.total_delay

    self.packets_failed += 1
    return None, None

def create_realistic_network():
    """Create a realistic network topology"""
    network = NetworkGraph()

    # Create a more complex network (ISP-like topology)
    # Format: (node1, node2, bandwidth_Mbps, delay_ms, packet_loss)
    links = [
        # Core routers (high bandwidth, low delay)
        (0, 1, 100, 2, 0.001),
        (1, 2, 100, 2, 0.001),
        (2, 3, 100, 3, 0.001),
        (0, 3, 80, 4, 0.002),

        # Distribution layer
        (1, 4, 50, 5, 0.01),
        (2, 5, 50, 5, 0.01),
        (3, 6, 50, 4, 0.01),
        (4, 5, 40, 6, 0.015),
        (5, 6, 40, 6, 0.015),

        # Access layer (lower bandwidth, higher delay)
        (4, 7, 20, 10, 0.02),
        (5, 8, 20, 12, 0.02),
        (6, 9, 20, 10, 0.02),
        (7, 8, 10, 15, 0.03),
        (8, 9, 10, 15, 0.03),

        # Backup paths
        (0, 4, 30, 8, 0.015),
        (3, 9, 30, 8, 0.015),
    ]

    for node1, node2, bw, delay, loss in links:
        network.add_link(node1, node2, bw, delay, loss)

    return network

def visualize_network_state(network, packets=None, highlight_path=None):
    """Visualize network with current state"""
    G = nx.Graph()

```

```

# Add edges with current load info
edge_colors = []
edge_widths = []

for (node1, node2), link in network.links.items():
    if node1 < node2:
        G.add_edge(node1, node2)
        # Color based on utilization
        utilization = link.current_load / link.bandwidth if link.bandwidth > 0 else 0

        if link.failed:
            edge_colors.append('black')
            edge_widths.append(1)
        elif utilization > 0.8:
            edge_colors.append('red')
            edge_widths.append(4)
        elif utilization > 0.5:
            edge_colors.append('orange')
            edge_widths.append(3)
        else:
            edge_colors.append('green')
            edge_widths.append(2)

pos = nx.spring_layout(G, seed=42, k=0.5)

plt.figure(figsize=(14, 10))

# Draw edges
nx.draw_networkx_edges(G, pos, edge_color=edge_colors, width=edge_widths, alpha=0.6)

# Highlight specific path if provided
if highlight_path:
    path_edges = [(highlight_path[i], highlight_path[i+1])
                  for i in range(len(highlight_path)-1)]
    nx.draw_networkx_edges(G, pos, edgelist=path_edges,
                          edge_color='blue', width=5, alpha=0.9, style='dashed')

# Draw nodes with size based on queue length
node_sizes = []
for node_id in G.nodes():
    queue_len = network.nodes[node_id].get_queue_length()
    node_sizes.append(700 + queue_len * 50)

nx.draw_networkx_nodes(G, pos, node_color='lightblue',
                      node_size=node_sizes, alpha=0.9)
nx.draw_networkx_labels(G, pos, font_size=12, font_weight='bold')

# Add legend
legend_elements = [
    plt.Line2D([0], [0], color='green', linewidth=3, label='Low Load (<50%)'),
    plt.Line2D([0], [0], color='orange', linewidth=3, label='Medium Load (50-80%)'),
    plt.Line2D([0], [0], color='red', linewidth=3, label='High Load (>80%)'),
    plt.Line2D([0], [0], color='black', linewidth=3, label='Failed Link'),
    plt.Line2D([0], [0], color='blue', linewidth=3, linestyle='dashed', label='Active Route')
]
plt.legend(handles=legend_elements, loc='upper left', fontsize=10)

```

```

plt.title("Network State (Node size = Queue length, Edge color = Utilization)",
        fontsize=14, fontweight='bold')
plt.axis('off')
plt.tight_layout()
plt.show()

def visualize_metrics(path_metrics, source, destination):
    """Visualize routing metrics over time"""
    fig, axes = plt.subplots(2, 2, figsize=(14, 10))

    metrics = path_metrics[(source, destination)]
    iterations = range(len(metrics))

    delays = [m['delay'] for m in metrics]
    bandwidths = [m['bandwidth'] for m in metrics]
    costs = [m['cost'] for m in metrics]

    # Delay over time
    axes[0, 0].plot(iterations, delays, 'b-', linewidth=2)
    axes[0, 0].set_xlabel('Packet Number')
    axes[0, 0].set_ylabel('End-to-End Delay (ms)')
    axes[0, 0].set_title('Path Delay Over Time')
    axes[0, 0].grid(True, alpha=0.3)

    # Bandwidth over time
    axes[0, 1].plot(iterations, bandwidths, 'g-', linewidth=2)
    axes[0, 1].set_xlabel('Packet Number')
    axes[0, 1].set_ylabel('Bottleneck Bandwidth (Mbps)')
    axes[0, 1].set_title('Available Bandwidth Over Time')
    axes[0, 1].grid(True, alpha=0.3)

    # Cost over time
    axes[1, 0].plot(iterations, costs, 'r-', linewidth=2)
    axes[1, 0].set_xlabel('Packet Number')
    axes[1, 0].set_ylabel('Path Cost')
    axes[1, 0].set_title('Routing Cost Over Time')
    axes[1, 0].grid(True, alpha=0.3)

    # Statistics
    axes[1, 1].axis('off')
    stats_text = f"""
Routing Statistics:

Average Delay: {sum(delays)/len(delays):.2f} ms
Min Delay: {min(delays):.2f} ms
Max Delay: {max(delays):.2f} ms

Average Bandwidth: {sum(bandwidths)/len(bandwidths):.2f} Mbps
Min Bandwidth: {min(bandwidths):.2f} Mbps

Average Cost: {sum(costs)/len(costs):.2f}

Total Packets: {len(metrics)}
"""
    axes[1, 1].text(0.1, 0.5, stats_text, fontsize=12, family='monospace',
                    verticalalignment='center')

```

```

plt.suptitle(f'Routing Metrics: Node {source} → Node {destination}',
            fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# Main simulation
if __name__ == "__main__":
    print("=" * 70)
    print("Advanced Ant Colony Optimization for Dynamic Network Packet Routing")
    print("=" * 70)

    # Create network
    network = create_realistic_network()
    print(f"\nNetwork created with {len(network.nodes)} nodes")
    print(f"Total links: {len(network.links) // 2}")

    # Initialize ACO router
    router = ACOPacketRouter(
        network=network,
        alpha=1.0,
        beta=2.5,
        evaporation=0.15,
        q_constant=100,
        num_ants=15
    )

    print("\n" + "=" * 70)
    print("Scenario 1: Normal Traffic")
    print("=" * 70)

    # Simulate packet routing
    source = 0
    destination = 9
    num_packets = 30

    print(f"\nRouting {num_packets} packets: Node {source} → Node {destination}")

    for i in range(num_packets):
        # Create packet with varying requirements
        priority = random.randint(1, 5)
        size = random.uniform(0.5, 3.0) # MB
        bandwidth_req = random.uniform(0, 10) # Mbps

        packet = DataPacket(i, source, destination, size, priority, bandwidth_req)
        path, delay = router.route_packet(packet)

        # Decay traffic on all links
        network.update_link_loads(decay_rate=0.2)

        if i % 10 == 0 and path:
            print(f'Packet {i}: Path = {' → '.join(map(str, path))}, "
                  f'Delay = {delay:.2f}ms, Priority = {priority}')

    print(f"\nPackets successfully routed: {router.packets_routed}")
    print(f"Packets failed: {router.packets_failed}")

    # Visualize current state

```

```

best_path, _, _ = router.best_paths.get((source, destination), (None, None, None))
print("\nGenerating network state visualization...")
visualize_network_state(network, highlight_path=best_path)

print("\n" + "=" * 70)
print("Scenario 2: Link Failure and Recovery")
print("=" * 70)

# Simulate link failure
print("\nSimulating link failure (1 → 2)...")
network.simulate_link_failure(1, 2)

for i in range(10):
    packet = DataPacket(num_packets + i, source, destination, 1.0, 3, 5)
    path, delay = router.route_packet(packet)
    network.update_link_loads(decay_rate=0.2)

    if path:
        print(f"Packet {num_packets + i}: Rerouted via {' → '.join(map(str, path))}")

print("\nRestoring failed link...")
network.restore_link(1, 2)

for i in range(10):
    packet = DataPacket(num_packets + 10 + i, source, destination, 1.0, 3, 5)
    path, delay = router.route_packet(packet)
    network.update_link_loads(decay_rate=0.2)

# Final visualizations
print("\nGenerating performance metrics...")
visualize_metrics(router.path_metrics, source, destination)
print("\n" + "=" * 70)
print("Simulation Complete!")
print("=" * 70)
print(f"Total packets routed: {router.packets_routed}")
print(f"Success rate: {(router.packets_routed/(router.packets_routed + router.packets_failed)*100):.1f}%")

```

Output:

Advanced Ant Colony Optimization for Dynamic Network Packet Routing

Network created with 10 nodes
Total links: 16

Scenario 1: Normal Traffic

Routing 30 packets: Node 0 → Node 9
Packet 0: Path = 0 → 1 → 2 → 3 → 9, Delay = 15.00ms, Priority = 1
Packet 10: Path = 0 → 1 → 2 → 3 → 6 → 5 → 4 → 7 → 8 → 9, Delay = 64.70ms, Priority = 5
Packet 20: Path = 0 → 1 → 2 → 3 → 6 → 5 → 4 → 7 → 8 → 9, Delay = 76.68ms, Priority = 3

Packets successfully routed: 20
Packets failed: 10

PROGRAM 5: CUCKOO SEARCH (CS)

Cuckoo Search (CS) is a nature-inspired optimization algorithm based on the brood parasitism of some cuckoo species. This behavior involves laying eggs in the nests of other birds, leading to the optimization of survival strategies. CS uses Lévy flights to generate new solutions, promoting global search capabilities and avoiding local minima. The algorithm is widely used for solving continuous optimization problems and has applications in various domains, including engineering design, machine learning, and data mining.

Algorithm:

1. $Best = a$
 $cost = 1$
 $K = 10$

2. Cuckoo Search Algorithm :-
 Step 1: Initialization values
 n ← population size
 p ← discovery probability
 max ← maximum number of iterations
 $bounds$ ← search bounds

Step 2: Initialize population
 For $i = 1$ to n :-
 initialize nest i with $a \leftarrow \text{rand}(\text{bounds})$
 Evaluate $Fitness_i(a)$
 [END FOR]

Step 3: Find current best solution
 $best \leftarrow \max(F_i)$

Step 4: Iteration until Stopping criterion
 For $t = 1$ to max :-
 For each $i = 1$ to n :-
 generate sol_{new} by Lévy flight
 $sol_{new} = \text{best sol} + a \cdot \text{rand}()$
 Fitness $F_{new} \leftarrow F(sol_{new})$
 $next_i \leftarrow \text{rand}(\text{bounds})$
 if $F_{new} < F_i$:-
 replace $next_i$ with sol_{new}
 End if
 End for
 For each $next_i$:-
 with probability p :-
 generate sol with a new $sol(\text{bounds})$
 End for
 find $best = \max(F_i)$

Return $best$

Fitness ($next_i$) :-
 $next_i$ - $next_i$ / ideal sol.

Optimization of Cuckoo Search for Smart Grid Load Scheduling

Output:-
 Cuckoo Search for Smart Grid Load Scheduling

Fitness of Naive Schedule : 7.39
 (Running at earliest possible time)

Running Cuckoo Search Optimizer :-

Iteration	10/100	20/100	100/100
Initial best Schedule cost	6.93		
Iteration 10/100	6.55		
Iteration 20/100	6.52		
Iteration 100/100			6.05

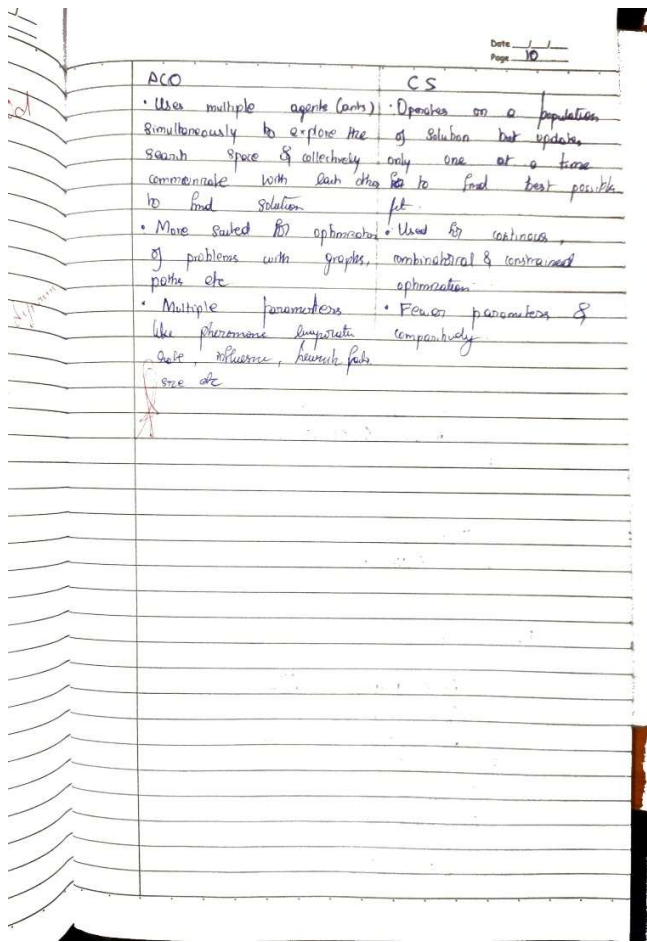
Optimized Schedule Cost : 6.05

Optimal Appliance schedule :-

Appliance	Start	End
Washing Machine	01:00	02:00
Dishwasher	21:00	23:00
EV charger	01:00	06:00
Water heater	06:00	07:00
pool pump	00:00	01:00

Total load allocation makes : 1.34

CS



Code:

```
import numpy as np
import random
import math
```

--- 1. PROBLEM DEFINITION: Smart Grid Environment ---

Define appliances with their power consumption (in kW) and required runtime (in hours)

and the window of time (hour of day, 0-23) they are allowed to run.

APPLIANCES = {

```
    'Washing Machine': {'power': 2.0, 'runtime': 2, 'window': (0, 6)}, # Can run between midnight and 6 AM
    'Dishwasher':      {'power': 1.5, 'runtime': 3, 'window': (20, 23)}, # Can run between 8 PM and 11 PM
    'EV Charger':       {'power': 7.0, 'runtime': 4, 'window': (22, 6)}, # Can run between 10 PM and 6 AM (overnight)
    'Water Heater':     {'power': 4.0, 'runtime': 1, 'window': (3, 7)},   # Can run between 3 AM and 7 AM
    'Pool Pump':        {'power': 1.0, 'runtime': 5, 'window': (0, 23)}, # Can run anytime
}
```

Time-of-Use (TOU) electricity pricing (cost per kWh)

Prices are higher during peak demand hours.

PRICE_TARIFF = {

```
    (0, 6): 0.12, # Off-Peak: 12 AM - 6 AM
    (7, 17): 0.20, # Mid-Peak: 7 AM - 5 PM
    (18, 21): 0.35, # On-Peak: 6 PM - 9 PM
```

```

    (22, 23): 0.20, # Mid-Peak: 10 PM - 11 PM
}

def get_price(hour):
    """Returns the electricity price for a given hour."""
    for (start, end), price in PRICE_TARIFF.items():
        if start <= hour <= end:
            return price
    return float('inf') # Should not happen with a complete tariff

def calculate_cost(schedule):
    """
    Fitness function: Calculates the total electricity cost for a given schedule.
    A 'schedule' is a list of integer start times for each appliance.
    """
    total_cost = 0
    appliance_names = list(APPLIANCES.keys())

    for i, start_time in enumerate(schedule):
        appliance = APPLIANCES[appliance_names[i]]
        power = appliance['power']
        runtime = appliance['runtime']

        # Calculate cost for the duration the appliance runs
        for hour_offset in range(runtime):
            # Use modulo 24 to handle schedules that wrap around midnight
            current_hour = (start_time + hour_offset) % 24
            total_cost += power * get_price(current_hour)

    return total_cost

def generate_random_schedule():
    """Generates a single valid random schedule (a nest)."""
    schedule = []
    for name, details in APPLIANCES.items():
        start_window, end_window = details['window']
        runtime = details['runtime']

        if start_window <= end_window:
            # Simple window (e.g., 8 AM to 5 PM)
            possible_starts = list(range(start_window, end_window - runtime + 2))
        else:
            # Window wraps around midnight (e.g., 10 PM to 6 AM)
            possible_starts = list(range(start_window, 24 - runtime + 1)) + list(range(0, end_window - runtime + 2))

        if not possible_starts:
            # Fallback if runtime is too long for the window
            possible_starts = [start_window]

        start_time = random.choice(possible_starts)
        schedule.append(start_time)

    return schedule

# --- 2. CUCKOO SEARCH ALGORITHM IMPLEMENTATION ---

def levy_flight(beta=1.5):

```

```

"""
Generate a step length from a Levy distribution.
This simulates the random walk pattern of cuckoos.
"""
sigma = (math.gamma(1 + beta) * math.sin(math.pi * beta / 2) /
          (math.gamma((1 + beta) / 2) * beta * 2**((beta - 1) / 2)))**(1 / beta)
u = np.random.normal(0, sigma)
v = np.random.normal(0, 1)
step = u / abs(v)**(1 / beta)
return step

def apply_bounds(new_start_time, appliance_name):
    """Ensure the new start time is valid for the appliance's window."""
    details = APPLIANCES[appliance_name]
    start_window, end_window = details['window']
    runtime = details['runtime']

    # Simple validation: wrap around using modulo and clamp if necessary
    # A more complex system might find the nearest valid time slot.
    new_start_time = int(round(new_start_time)) % 24

    if start_window <= end_window:
        if not (start_window <= new_start_time <= end_window - runtime + 1):
            return start_window # Reset to a known valid start
    else: # Wraps around midnight
        if not ((start_window <= new_start_time <= 23) or (0 <= new_start_time <= end_window - runtime + 1)):
            return start_window # Reset to a known valid start

    return new_start_time

def cuckoo_search_optimizer(n_nests, n_iterations, pa):
    """
    Main Cuckoo Search optimization function.

    :param n_nests: Number of nests in the population.
    :param n_iterations: Number of generations to run.
    :param pa: Fraction of nests to be abandoned and replaced.
    """
    # Initialize population of nests
    nests = [generate_random_schedule() for _ in range(n_nests)]
    costs = [calculate_cost(s) for s in nests]

    best_nest_index = np.argmin(costs)
    best_cost = costs[best_nest_index]
    best_schedule = nests[best_nest_index]

    print(f"Initial best schedule cost: ${best_cost:.2f}")

    # Main optimization loop
    for t in range(n_iterations):
        # 1. Get a cuckoo (generate a new solution via Levy flights)
        # Randomly choose a nest to modify
        i = random.randint(0, n_nests - 1)
        current_nest = nests[i][:]

        # Generate new solution

```

```

step_size = 0.1 # A scaling factor for the levy flight step
new_nest = current_nest[:]

# Modify one appliance's start time in the chosen nest
app_to_modify = random.randint(0, len(APPLIANCES) - 1)
app_name = list(APPLIANCES.keys())[app_to_modify]

new_start = new_nest[app_to_modify] + step_size * levy_flight()
new_nest[app_to_modify] = apply_bounds(new_start, app_name)

new_cost = calculate_cost(new_nest)

# 2. Compare the new solution with a randomly chosen existing one
j = random.randint(0, n_nests - 1)
if new_cost < costs[j]:
    nests[j] = new_nest
    costs[j] = new_cost

# 3. Abandon a fraction (pa) of the worst nests and build new ones
n_abandon = int(pa * n_nests)
worst_indices = np.argsort(costs)[-n_abandon:]

for idx in worst_indices:
    nests[idx] = generate_random_schedule()
    costs[idx] = calculate_cost(nests[idx])

# 4. Find the current best solution
current_best_idx = np.argmin(costs)
if costs[current_best_idx] < best_cost:
    best_cost = costs[current_best_idx]
    best_schedule = nests[current_best_idx]

if (t + 1) % 10 == 0:
    print(f'Iteration {t+1}/{n_iterations} | Best Cost: ${best_cost:.2f}')

return best_schedule, best_cost

# --- 3. RUN SIMULATION ---
if __name__ == "__main__":
    # Cuckoo Search Parameters
    NUM_NESTS = 50
    ITERATIONS = 100
    ABANDON_FRACTION = 0.25

    print("--- Cuckoo Search for Smart Grid Load Scheduling ---")

    # Calculate cost of a non-optimized, naive schedule (running everything as early as possible)
    naive_schedule = []
    for name, details in APPLIANCES.items():
        naive_schedule.append(details['window'][0])
    naive_cost = calculate_cost(naive_schedule)
    print(f"\nCost of a NAIVE schedule (running at earliest possible time): ${naive_cost:.2f}\n")

    print("Running Cuckoo Search Optimizer...")

    optimized_schedule, optimized_cost = cuckoo_search_optimizer(
        n_nests=NUM_NESTS,

```

```

        n_iterations=ITERATIONS,
        pa=ABANDON_FRACTION
    )

    print("\n--- Optimization Complete ---")
    print(f"Optimized Schedule Cost: ${optimized_cost:.2f}")

    print("\nOptimal Appliance Schedule:")
    appliance_names = list(APPLIANCES.keys())
    for i, start_time in enumerate(optimized_schedule):
        appliance_name = appliance_names[i]
        runtime = APPLIANCES[appliance_name]['runtime']
        end_time = (start_time + runtime - 1) % 24
        print(f"- {appliance_name}: Start at {start_time:02d}:00, End at {end_time:02d}:00")

    print(f"\nTotal savings compared to naive schedule: ${naive_cost - optimized_cost:.2f}")

```

Output:

--- Cuckoo Search for Smart Grid Load Scheduling ---

Cost of a NAIVE schedule (running at earliest possible time): \$7.39

Running Cuckoo Search Optimizer...

Initial best schedule cost: \$6.05

Iteration 10/100 | Best Cost: \$6.05

Iteration 20/100 | Best Cost: \$6.05

Iteration 30/100 | Best Cost: \$6.05

Iteration 40/100 | Best Cost: \$6.05

Iteration 50/100 | Best Cost: \$6.05

Iteration 60/100 | Best Cost: \$6.05

Iteration 70/100 | Best Cost: \$6.05

Iteration 80/100 | Best Cost: \$6.05

Iteration 90/100 | Best Cost: \$6.05

Iteration 100/100 | Best Cost: \$6.05

--- Optimization Complete ---

Optimized Schedule Cost: \$6.05

Optimal Appliance Schedule:

- Washing Machine : Start at 01:00, End at 02:00

- Dishwasher : Start at 21:00, End at 23:00

- EV Charger : Start at 01:00, End at 04:00

- Water Heater : Start at 06:00, End at 06:00

- Pool Pump : Start at 00:00, End at 04:00

Total savings compared to naive schedule: \$1.34

PROGRAM 6: GREY WOLF OPTIMIZER (GWO)

The Grey Wolf Optimizer (GWO) algorithm is a swarm intelligence algorithm inspired by the social hierarchy and hunting behavior of grey wolves. It mimics the leadership structure of alpha, beta, delta, and omega wolves and their collaborative hunting strategies. The GWO algorithm uses these social hierarchies to model the optimization process, where the alpha wolves guide the search process while beta and delta wolves assist in refining the search direction. This algorithm is effective for continuous optimization problems and has applications in engineering, data analysis, and machine learning.

Algorithm:

The image shows a handwritten algorithm for the Grey Wolf Optimizer (GWO) on a two-page spread of a notebook. The left page contains the high-level algorithm steps, and the right page contains the detailed implementation for each wolf.

Left Page (Algorithm Overview):

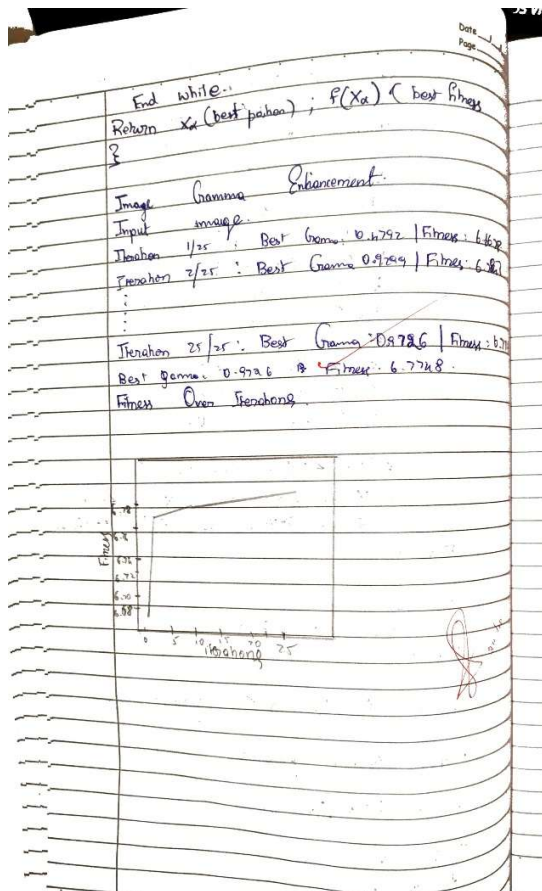
```

(A) Grey Wolf Optimizer
Algorithm
input: f
objective function  $f(x) \leftarrow$  to be minimized/maximized
bounds  $\leftarrow [x_{min}, x_{max}]$ 
 $N \leftarrow$  no. of wolves
 $T_{max} \leftarrow$  maximum iterations
§
Initialize population: §
for each wolf  $i = 1$  to  $N$ :
    initialize  $X_i \leftarrow [x_{min}, x_{max}]$ 
End for
§
Fitness  $F(X_i)$  §
return:  $F(X_i)$  / optimum solution
§
Evaluate fitness  $F(X_i)$  for each wolf
Initialize  $X_\alpha \leftarrow$  best solution (highest  $F(X_i)$ )
 $X_\beta \leftarrow$  2nd best solution
 $X_\delta \leftarrow$  3rd best solution
 $c_1, c_2, c_3 \leftarrow \alpha, \beta, \delta$ 
set iteration counter  $t \leftarrow 1$ 
control parameter  $a \leftarrow 2$ 
while ( $t \leq T_{max}$ ) §
    
```

Right Page (Detailed Implementation):

```

for each wolf  $i = 1$  to  $N$ :
     $r_1, r_2 \in [0, 1]$ 
    compute  $A = 2 \cdot a \cdot r_1 - a$ 
     $C = 2 \cdot r_2$ 
     $D_\alpha \leftarrow |C \cdot X_\alpha - X_i|$ 
     $D_\beta \leftarrow |C \cdot X_\beta - X_i|$ 
     $D_\delta \leftarrow |C \cdot X_\delta - X_i|$ 
     $X_1 \leftarrow X_i - A_1 \cdot D_\alpha$ 
     $X_2 \leftarrow X_i - A_2 \cdot D_\beta$ 
     $X_3 \leftarrow X_i - A_3 \cdot D_\delta$ 
     $X_i(t+1) \leftarrow (X_1 + X_2 + X_3) / 3$ 
    if  $X_i(t+1) < x_{min}$  then  $X_i(t+1) \leftarrow x_{min}$ 
    if  $X_i(t+1) > x_{max}$  then  $X_i(t+1) \leftarrow x_{max}$ 
End for
for each wolf  $i$ :
    Evaluate fitness  $F(X_i)$ 
End for
Update  $X_\alpha, X_\beta, X_\delta \leftarrow$  sorted( $F(X_i)$ )
// Update control parameters
 $a = 2 - (2 \cdot t / T_{max})$ 
// Increment
 $t = t + 1$ 
    
```



Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# -----
# Step 1: Load grayscale image
# -----
img = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
img = cv2.normalize(img, None, 0, 1, cv2.NORM_MINMAX, dtype=cv2.CV_32F)

# -----
# Step 2: Fitness function
# -----
def fitness_function(gamma):
    gamma = np.clip(gamma, 0.1, 5.0)
    enhanced = np.power(img, gamma)
    hist = np.histogram(enhanced, bins=256, range=(0, 1))[0]
    hist = hist / np.sum(hist)
    entropy = -np.sum(hist * np.log2(hist + 1e-12))
    return entropy # Maximize entropy

# -----
```

```

# Step 3: Grey Wolf Optimizer (GWO) with iteration tracking
# -----
def GWO(obj_func, wolves=10, max_iter=20, dim=1, lb=0.1, ub=5.0):
    positions = np.random.uniform(lb, ub, (wolves, dim))

    alpha_pos, beta_pos, delta_pos = np.zeros(dim), np.zeros(dim), np.zeros(dim)
    alpha_score, beta_score, delta_score = -np.inf, -np.inf, -np.inf

    iteration_history = [] # store best fitness each iteration

    for t in range(max_iter):
        for i in range(wolves):
            fitness = obj_func(positions[i])

            if fitness > alpha_score:
                delta_score, delta_pos = beta_score, beta_pos.copy()
                beta_score, beta_pos = alpha_score, alpha_pos.copy()
                alpha_score, alpha_pos = fitness, positions[i].copy()
            elif fitness > beta_score:
                delta_score, delta_pos = beta_score, beta_pos.copy()
                beta_score, beta_pos = fitness, positions[i].copy()
            elif fitness > delta_score:
                delta_score, delta_pos = fitness, positions[i].copy()

        # Update positions
        a = 2 - t * (2 / max_iter)
        for i in range(wolves):
            for j in range(dim):
                r1, r2 = np.random.rand(), np.random.rand()
                A1 = 2 * a * r1 - a
                C1 = 2 * r2
                D_alpha = abs(C1 * alpha_pos[j] - positions[i][j])
                X1 = alpha_pos[j] - A1 * D_alpha

                r1, r2 = np.random.rand(), np.random.rand()
                A2 = 2 * a * r1 - a
                C2 = 2 * r2
                D_beta = abs(C2 * beta_pos[j] - positions[i][j])
                X2 = beta_pos[j] - A2 * D_beta

                r1, r2 = np.random.rand(), np.random.rand()
                A3 = 2 * a * r1 - a
                C3 = 2 * r2
                D_delta = abs(C3 * delta_pos[j] - positions[i][j])
                X3 = delta_pos[j] - A3 * D_delta

                positions[i][j] = np.clip((X1 + X2 + X3) / 3, lb, ub)

        # Save iteration info
        iteration_history.append(alpha_score)
        print(f'Iteration {t+1}/{max_iter} | Best Gamma: {alpha_pos[0]:.4f} | Fitness: {alpha_score:.4f}')

    return alpha_pos, alpha_score, iteration_history

# Step 4: Run GWO
# -----
best_gamma, best_score, history = GWO(fitness_function, wolves=10, max_iter=25)

```

```

print(f"\nFinal Best gamma: {best_gamma[0]:.4f}, Fitness: {best_score:.4f}")

# -----
# Step 5: Enhance image
# -----
enhanced_img = np.power(img, best_gamma[0])

# -----
# Step 6: Plot results
plt.figure(figsize=(12,5))
plt.subplot(1,3,1)
plt.imshow(img, cmap='gray')
plt.title("Original")

plt.subplot(1,3,2)
plt.imshow(enhanced_img, cmap='gray')
plt.title(f"Enhanced ( $\gamma={best\_gamma[0]:.3f}$ ")

plt.subplot(1,3,3)
plt.plot(history, 'r-', linewidth=2)
plt.title("Fitness over Iterations")
plt.xlabel("Iteration")
plt.ylabel("Fitness (Entropy)")

plt.tight_layout()
plt.show()

```

Output:

Iteration 1/25	Best Gamma: 0.4792	Fitness: 6.6668
Iteration 2/25	Best Gamma: 0.9299	Fitness: 6.7748
Iteration 3/25	Best Gamma: 0.9299	Fitness: 6.7748
Iteration 4/25	Best Gamma: 0.9726	Fitness: 6.7748
Iteration 5/25	Best Gamma: 0.9726	Fitness: 6.7748
Iteration 6/25	Best Gamma: 0.9726	Fitness: 6.7748
Iteration 7/25	Best Gamma: 0.9726	Fitness: 6.7748
Iteration 8/25	Best Gamma: 0.9726	Fitness: 6.7748
Iteration 9/25	Best Gamma: 0.9726	Fitness: 6.7748
Iteration 10/25	Best Gamma: 0.9726	Fitness: 6.7748
Iteration 11/25	Best Gamma: 0.9726	Fitness: 6.7748
Iteration 12/25	Best Gamma: 0.9726	Fitness: 6.7748
Iteration 13/25	Best Gamma: 0.9726	Fitness: 6.7748
Iteration 14/25	Best Gamma: 0.9726	Fitness: 6.7748
Iteration 15/25	Best Gamma: 0.9726	Fitness: 6.7748
Iteration 16/25	Best Gamma: 0.9726	Fitness: 6.7748
Iteration 17/25	Best Gamma: 0.9726	Fitness: 6.7748
Iteration 18/25	Best Gamma: 0.9726	Fitness: 6.7748
Iteration 19/25	Best Gamma: 0.9726	Fitness: 6.7748
Iteration 20/25	Best Gamma: 0.9726	Fitness: 6.7748
Iteration 21/25	Best Gamma: 0.9726	Fitness: 6.7748
Iteration 22/25	Best Gamma: 0.9726	Fitness: 6.7748
Iteration 23/25	Best Gamma: 0.9726	Fitness: 6.7748
Iteration 24/25	Best Gamma: 0.9726	Fitness: 6.7748
Iteration 25/25	Best Gamma: 0.9726	Fitness: 6.7748

Final Best gamma: 0.9726, Fitness: 6.7748

PROGRAM 7: PARALLEL CELLULAR ALGORITHM AND PROGRAMS

Parallel Cellular Algorithms are inspired by the functioning of biological cells that operate in a highly parallel and distributed manner. These algorithms leverage the principles of cellular automata and parallel computing to solve complex optimization problems efficiently. Each cell represents a potential solution and interacts with its neighbors to update its state based on predefined rules. This interaction models the diffusion of information across the cellular grid, enabling the algorithm to explore the search space effectively. Parallel Cellular Algorithms are particularly suitable for large-scale optimization problems and can be implemented on parallel computing architectures for enhanced performance.

Algorithm:

(B) Parallel Cellular Algorithm

Algorithm :-

input () :

- $N \leftarrow \text{population_size}$
- $N(c) \leftarrow \text{neighborhood_structure}$
- $F(c) \leftarrow \text{Fitness_function}$
- $\text{Max_iter} \leftarrow \text{Maximum_iterations}$
- $\text{Update}(C, \text{best}) \leftarrow \text{Update_rule}$
- $\text{Term} \leftarrow \text{termination_criteria}$

Step 1: Initialization

- array $P[N] \leftarrow \{c_1, c_2, \dots, c_N\}$
- for each cell C_i :
- $C_i \leftarrow \text{Possible_random_solution}$
- evaluate $\text{Fitness}[i] \leftarrow F(C_i)$
- $C\text{-global_best} \leftarrow \max(\text{Fitness}[i])$

Step 2 :-

In parallel for each cell C_i in P do :

- $N_i \leftarrow N(C_i)$
- $C_{\text{best}} \leftarrow \max(F(c_j) \mid F(c_j) \in N_i)$
- $C_{i\text{-new}} \leftarrow \text{Update}(C_i, C_{\text{best}})$
- $\text{Fitness_new}[i] \leftarrow F(C_{i\text{-new}})$

End parallel for

Step 3 :- Synchronization

For each cell C_i :

- if $F(C_{i\text{-new}}) \geq F(C_i)$:
- $F(C_i) \leftarrow F(C_{i\text{-new}})$
- $C_i \leftarrow C_{i\text{-new}}$

$\text{Fitness}[i] \leftarrow \text{Fitness_new}[i]$

Step 4 :

- Update $C\text{-global_best} \leftarrow \max(\text{Fitness}[i])$
- if condition "Term" then :
- Exit Loop

End For

Steps :- Output

- return $C\text{-global_best}$

Update rule :- {

- problem specific logic :
- solves best fit

}

~~Routing~~

~~Resource Allocation~~

Resource Allocation routing

Starting point

Node 1 : 0.125

2 : 0.125

3 : 0.125

4 : 0.125

5 : 0.125

6 : 0.125

7 : 0.125

8 : 0.125

Initial fitness = 0.000

Iteration 1/40 | Best fitness : 0.0125

Iteration 2/40 | Best fitness : 0.0192

...

Iteration 40/40 | Best fitness : 0.0199

Date: / /
Page:

Final Optimized Resource Allocation:	
Node 1:	0.134
Node 2:	0.108
Node 3:	0.108
Node 4:	0.124
Node 5:	0.113
Node 6:	0.129
Node 7:	0.128
Node 8:	0.126
Total Allocated Resource: 1.0000	
Final Fitness: 0.0000	
Old: From cells to islands: A unified model of PCA Old PCA vs New PCA	
Problem Complexity: Simple Topology: Fixed, simple grid Parallelism: Shared memory, limited cores Real world: Limited scalability, academic Theory: Focus on parallel performance Complexity: relatively simpler Time: $O(P \times T \times Nnb)$ $P \rightarrow$ no. of cells $T \rightarrow$ no. of iterations $Nnb \rightarrow$ neighborhood	Problem Complexity: Multi Objective Topology: Adaptive, complex, hybrid Parallelism: Multi core, GPUs, dynamic load balancing Real world: More complex real world problems Theory: More on convergence, adaptive multi objective theory Complexity: more complex tuning Time: $O(P \times T \times (Feval + Nnb))$ $Feval$ - eval function / fitness but with asynchronous or vectorized updates for highly reduced time complexity

Code:

```
import numpy as np
from multiprocessing import Pool
import matplotlib.pyplot as plt

# -----
# Defined Inputs
# -----
num_nodes = 8          # Number of network nodes
total_resource = 1.0    # Total available resource
grid_size = (6, 6)     # PCA grid (6x6 = 36 cells)
iterations = 40         # Optimization iterations

# Network topology (simple ring)
neighbor_links = [(i, (i+1) % num_nodes) for i in range(num_nodes)]

# -----
# Objective Function
# -----
def fitness(solution):
```

```

"""
Objective: Maximize - (imbalance + routing_penalty)
"""
solution = np.clip(solution, 0, 1)
solution = solution / np.sum(solution) # Normalize so total = 1

imbalance = np.var(solution)
routing_penalty = sum(abs(solution[i] - solution[j]) for i, j in neighbor_links) / len(neighbor_links)

return -(imbalance + routing_penalty) # Maximize fitness

# -----
# Mutation (local variation)
# -----
def mutate(solution, rate=0.1):
    new_sol = solution.copy()
    idx = np.random.randint(len(solution))
    new_sol[idx] += np.random.uniform(-rate, rate)
    new_sol = np.clip(new_sol, 0, 1)
    new_sol /= np.sum(new_sol)
    return new_sol

# -----
# Local Evolution Function
# -----
def evolve_cell(args):
    i, j, grid, neighborhood = args
    rows, cols = grid.shape[:2]

    # Get neighbors (Von Neumann)
    neighbors = []
    for di, dj in neighborhood:
        ni, nj = (i + di) % rows, (j + dj) % cols
        neighbors.append(grid[ni, nj])

    # Compare with neighbors
    all_solutions = neighbors + [grid[i, j]]
    best_sol = max(all_solutions, key=fitness)

    # Apply mutation for diversity
    offspring = mutate(best_sol)
    return i, j, offspring

# -----
# Parallel Cellular Algorithm
# -----
def parallel_cellular_algorithm(grid_size=(6,6), sol_size=8, iterations=40, start_solution=None):
    grid = np.zeros((grid_size[0], grid_size[1], sol_size))

    # Initialize with small random variations around the starting point
    for i in range(grid.shape[0]):
        for j in range(grid.shape[1]):
            grid[i, j] = start_solution + np.random.uniform(-0.05, 0.05, sol_size)
            grid[i, j] = np.clip(grid[i, j], 0, 1)
            grid[i, j] /= np.sum(grid[i, j])

    neighborhood = [(-1,0),(1,0),(0,-1),(0,1)]

```

```

best_scores = []

for it in range(iterations):
    args = [(i, j, grid, neighborhood) for i in range(grid.shape[0]) for j in range(grid.shape[1])]
    with Pool() as pool:
        results = pool.map(evolve_cell, args)

    for i, j, new_sol in results:
        grid[i, j] = new_sol

    # Record best fitness this iteration
    all_fitness = np.array([[fitness(grid[i, j]) for j in range(grid.shape[1])] for i in range(grid.shape[0])])
    best_scores.append(all_fitness.max())

    print(f"Iteration {it+1}/{iterations} | Best Fitness: {best_scores[-1]:.4f}")

# Return best solution
best_idx = np.unravel_index(np.argmax(all_fitness), all_fitness.shape)
best_solution = grid[best_idx]

return best_solution, best_scores

# -----
# Starting Point
# -----
start_solution = np.ones(num_nodes) * (total_resource / num_nodes)
print("❏ Starting Point (Equal Allocation):")
for i, val in enumerate(start_solution):
    print(f"Node {i+1}: {val:.3f}")

start_fitness = fitness(start_solution)
print(f"\nInitial Fitness: {start_fitness:.4f}\n")

# -----
# Run the Algorithm
# -----
best_sol, history = parallel_cellular_algorithm(grid_size, num_nodes, iterations, start_solution)

# -----
# Results
# -----
print("\n✅ Final Optimized Resource Allocation:")
for i, val in enumerate(best_sol):
    print(f"Node {i+1}: {val:.3f}")

print(f"\nTotal Allocated Resource = {np.sum(best_sol):.3f}")
print(f"Final Fitness: {fitness(best_sol):.4f}")

# -----
# Visualization
# -----
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.plot(history, 'r-', linewidth=2)
plt.title("Fitness Over Iterations")
plt.xlabel("Iteration")

```

```

plt.ylabel("Best Fitness")

plt.subplot(1,2,2)
x = np.arange(1, num_nodes+1)
plt.bar(x-0.2, start_solution, width=0.4, label='Start', color='gray')
plt.bar(x+0.2, best_sol, width=0.4, label='Optimized', color='green')
plt.xlabel("Node")
plt.ylabel("Resource Share")
plt.title("Start vs Optimized Allocation")
plt.legend()
plt.tight_layout()
plt.show()

```

Output:

▣ Starting Point (Equal Allocation):

Node 1: 0.125
Node 2: 0.125
Node 3: 0.125
Node 4: 0.125
Node 5: 0.125
Node 6: 0.125
Node 7: 0.125
Node 8: 0.125

Initial Fitness: -0.0000

Iteration 1/40	Best Fitness: -0.0125
Iteration 2/40	Best Fitness: -0.0192
Iteration 3/40	Best Fitness: -0.0141
Iteration 4/40	Best Fitness: -0.0162
Iteration 5/40	Best Fitness: -0.0158
Iteration 6/40	Best Fitness: -0.0199
Iteration 7/40	Best Fitness: -0.0158
Iteration 8/40	Best Fitness: -0.0199
Iteration 9/40	Best Fitness: -0.0241
Iteration 10/40	Best Fitness: -0.0199
Iteration 11/40	Best Fitness: -0.0158
Iteration 12/40	Best Fitness: -0.0199
Iteration 13/40	Best Fitness: -0.0158
Iteration 14/40	Best Fitness: -0.0199
Iteration 15/40	Best Fitness: -0.0158
Iteration 16/40	Best Fitness: -0.0199
Iteration 17/40	Best Fitness: -0.0158
Iteration 18/40	Best Fitness: -0.0199
Iteration 19/40	Best Fitness: -0.0138
Iteration 20/40	Best Fitness: -0.0094
Iteration 21/40	Best Fitness: -0.0096
Iteration 22/40	Best Fitness: -0.0083
Iteration 23/40	Best Fitness: -0.0137
Iteration 24/40	Best Fitness: -0.0094
Iteration 25/40	Best Fitness: -0.0136
Iteration 26/40	Best Fitness: -0.0094
Iteration 27/40	Best Fitness: -0.0130
Iteration 28/40	Best Fitness: -0.0094
Iteration 29/40	Best Fitness: -0.0136

Iteration 30/40	Best Fitness: -0.0094
Iteration 31/40	Best Fitness: -0.0136
Iteration 32/40	Best Fitness: -0.0094
Iteration 33/40	Best Fitness: -0.0136
Iteration 34/40	Best Fitness: -0.0094
Iteration 35/40	Best Fitness: -0.0136
Iteration 36/40	Best Fitness: -0.0095
Iteration 37/40	Best Fitness: -0.0136
Iteration 38/40	Best Fitness: -0.0095
Iteration 39/40	Best Fitness: -0.0136
Iteration 40/40	Best Fitness: -0.0095

☒ Final Optimized Resource Allocation:

Node 1: 0.134
Node 2: 0.108
Node 3: 0.108
Node 4: 0.124
Node 5: 0.128
Node 6: 0.129
Node 7: 0.126
Node 8: 0.143

Total Allocated Resource = 1.000

Final Fitness: -0.0095

